

1.1 목차

1	강의소개	3
1.1	[Season 1] Spring AI 실전: 프리미엄 예약 AI 에이전트 개발	4
1.2	[Season 2] Spring AI 실전: 프리미엄 예약 AI 에이전트 개발	7
2	프리미엄 예약 AI 에이전트 'GourmetBot' 개발	10
2.1	프로젝트 개요.....	10
2.2	실습 시나리오 (User Stories).....	10
2.3	개발 환경 구성 (Project Setup).....	12
2.3.1	build.gradle 의존성 추가.....	13
2.3.2	application.properties 설정	13
2.4	도메인 모델링 및 DTO 설계	14
2.4.1	상수 정의 (Enums).....	14
2.4.2	엔티티 설계 (Entities)	15
2.4.3	DTO 설계 (Data Transfer Objects)	17
2.5	리포지토리(Repository) 구현	19
2.5.1	CustomerRepository: 고객 식별의 핵심	19
2.5.2	RestaurantTableRepository: 물리적 가용성 필터링	20
2.5.3	ReservationRepository: 시간 겹침 계산 (Logic Core)	20
2.6	서비스 계층 (Service) 구현.....	22
2.6.1	기본 설정 및 의존성 주입.....	22
2.6.2	[Logic 1] 빈 테이블 찾기 (Set Difference Algorithm)	23
2.6.3	[Logic 2] 예약 생성 및 CRM 연동 (Upsert Pattern).....	24
2.6.4	[Logic 3 & 4] 고객 조회 및 예약 취소.....	26

2.7	AI 도구(Tool) 구현.....	28
2.7.1	GourmetBotTools 클래스 구조.....	28
2.8	설정(Configuration) 및 데이터 초기화.....	31
2.8.1	AiConfig: 에이전트 조립 및 시공간 주입.....	31
2.8.2	DataLoader: 초기 데이터 생성 (World Building)	33
2.9	컨트롤러(Controller) 구현 및 테스트.....	34
2.9.1	AiController 구현.....	34
2.10	시스템 프롬프트 (System Prompt) 작성	36
2.11	최종 테스트 (Putting It All Together).....	40
2.11.1	서버 실행 및 데이터 확인.....	40
2.11.2	시나리오 테스트 (Postman / CURL).....	41
2.12	프로젝트 마무리	45
3	[부록] 실전 예약 테스트 시나리오	46
4	[부록] 실전 예약 취소 테스트 시나리오	51
5	[부록] 실전 예약 조회 테스트 시나리오	54

2 강의소개

Spring AI 실전 - 프리미엄 예약 AI 에이전트 'GourmetBot' 개발

목표 (Goal)

1. **Effective Agent 설계**: 위 아키텍처 다이어그램처럼 뇌-기억-도구가 분리된 견고한 시스템을 구현합니다.
2. **Tool Calling 마스터**: AI가 스스로 판단하여 Java 메서드(비즈니스 로직)를 호출하고 DB를 조작하게 만듭니다.
3. **비즈니스 로직 통합**: 단순 질의응답이 아닌, **조회(R) - 생성(C) - 취소(D)**가 유기적으로 연결된 완전한 백엔드 시스템을 구축합니다.

강의 동기 (Motivation)

현업에서는 단순한 대화형 AI를 넘어, ***실제 우리 회사의 DB와 연동되어 업무를 처리해 주는 에이전트***를 원합니다. 이 강의는 전통적인 **백엔드 기술(DB, 트랜잭션)**과 최신 **AI 기술(LLM, Tool Calling)**을 결합하여 실무적인 문제 해결 능력을 갖춘 에이전트를 만드는 방법을 알려드립니다.

컨셉 (Concept)

"GourmetBot(고메봇)" - 프리미엄 파인 다이닝의 AI 컨시어지

- Smart: 고객의 방문 이력을 기억하고 맞춤형 테이블을 제안합니다.
- Safe: 알레르기 정보를 필수적으로 체크하고, 데이터 무결성을 보장합니다.
- Polite: 기계적이지 않은 우아하고 격식 있는 톤앤매너를 구사합니다.

핵심 내용 (Core Content)

1. Spring AI 아키텍처 구현: 다이어그램의 각 요소를 코드로 구현 (System Prompt, Repository, Tools).
2. 고급 프롬프트 엔지니어링: Slot Filling(되묻기), Time Awareness(날짜 계산), Chain of Thought(단계적 사고) 기법 적용.
3. 데이터 무결성 & 트랜잭션: Upsert Pattern(자동 회원가입), Soft Delete(예약 취소 상태 관리) 구현.
4. Tool의 엄격성: Java Record(DTO)와 JSON Schema를 활용한 강력한 입력 데이터 검증.

2.1 [Season 1] Spring AI 실전: 프리미엄 예약 AI 에이전트 개발

목표: RDB(MySQL)와 연동되어 스스로 판단하고 데이터를 조작하는 'Effective Agent' 구축

강의 개요

- 난이도: 중급 (Spring Boot 기본기 필요)
 - 목표: RDB(MySQL)와 연동되어 스스로 판단하고 데이터를 조작하는 'AI 에이전트' 구축
 - 특징: '예약 조회' 기능을 과제(Challenge)로 남겨두어 수강생의 능동적 학습 유도
-

31 섹션 1. 오리엔테이션: 챗봇을 넘어 에이전트로

"말만 잘하는 AI는 필요 없다. 일을 하는 AI를 만들자!"

- 1-1. Intro & 데모 시연:
 - 완성된 GourmetBot 시연 (예약, 취소, 고객 식별).
 - 챗봇(Chatbot) vs 에이전트(Agent)의 결정적 차이 (Function Calling).
- 1-2. 우리가 만들 시스템 아키텍처:
 - LLM(두뇌) + Spring Boot(몸통) + MySQL(기억) 구조 설명.
 - [Diagram] 에이전트 아키텍처 다이어그램 활용.
- 1-3. 개발 환경 세팅:
 - Spring Initializr 프로젝트 생성.
 - 필수 의존성 추가 (Spring AI, JPA, MySQL, Lombok).
 - application.yml 설정 (OpenAI API Key, DB 연결).

31 섹션 2. 데이터 설계: 에이전트의 '기억' 만들기

"똑똑한 AI는 잘 정리된 데이터 위에서 탄생합니다."

- 2-1. 도메인 모델링 (Entity & Enum):

- 왜 테이블을 3개(Customer, Table, Reservation)로 쪼개야 하는가? (정규화).
- Enum(TableType, Status)을 통한 AI의 환각 방지 전략.
- **2-2. 리포지토리 구현 (Repository):**
 - JPA 기본 조회 메서드.
 - **[핵심]** 시간 겹침 계산 쿼리 (findBookedTableIds) 구현 원리 설명.

섹션 3. 비즈니스 로직: 에이전트의 '손발' 만들기

"AI는 계산을 못합니다. 자바가 대신 해줘야 합니다."

- **3-1. 서비스 계층 설계 (ReservationService):**
 - 빈 테이블 찾기 알고리즘 (차집합 연산).
- **3-2. CRM 자동화 (Upsert Pattern):**
 - "예약하면 자동 회원가입" 로직 구현 (get-or-create).
 - 알레르기 정보를 고객 메모에 영구 저장하는 센스.
- **3-3. 예약 취소 로직 (State Change):**
 - Soft Delete(CANCELLED)의 개념과 JPA Dirty Checking.
 - 모호성 해결을 위해 리스트 반환하기.

섹션 4. AI 연동: 뇌와 손발 연결하기 (Tool Calling)

"이제 자바 코드를 AI가 쓸 수 있는 도구로 변환합니다."

- **4-1. DTO 설계 (Java Record):**
 - @JsonPropertyDescription의 역할: AI에게 보내는 주석.
 - 입력 데이터의 엄격한 타입 정의.
- **4-2. 도구(Tool) 등록:**
 - GourmetBotTools 클래스 구현.
 - @Tool 어노테이션과 예외 처리(Self-Correction) 전략.

섹션 5. 프롬프트 엔지니어링: 에이전트의 '지능' 심기

"프롬프트는 자연어로 짜는 코드입니다."

- 5-1. 시스템 프롬프트 작성 (System Prompt):
 - 페르소나 정의 및 목표 설정.
 - [Time Injection] AI에게 '오늘 날짜'와 '요일' 알려주기.
- 5-2. 슬롯 필링 (Slot Filling) & 안전 점검:
 - 부족한 정보 되묻기 규칙 ("언제 오시나요?").
 - 이름 오입력 방지 ("고객님" 저장 금지).
 - 알레르기 체크 강제화.
- 5-3. AiConfig & Controller 구현:
 - SystemPromptTemplate 설정.
 - REST API 구현 및 Conversation-Id를 이용한 세션 유지.

섹션 6. 실전 테스트 & 디버깅

"만들었으면 대화해 봅시다. 과연 내 뜻대로 움직일까요?"

- 6-1. 데이터 초기화 (DataLoader):
 - 테스트를 위한 테이블 및 VIP 고객 자동 생성.
- 6-2. 시나리오 테스트 A (신규 예약):
 - 불친절한 고객("예약해줘") vs 똑똑한 AI("언제요?").
 - 알레르기 체크 및 예약 확정 확인.
- 6-3. 시나리오 테스트 B (예약 취소):
 - 취소 후 해당 테이블이 다시 검색되는지 검증.

섹션 7. [Challenge] 여러분의 차례입니다!

"보는 것만으로는 실력이 늘지 않습니다. 직접 구현해 보세요."

- 7-1. 도전 과제 소개:
 - 미션: 홍길동 고객이 "내 예약 보여줘"라고 했을 때, 예약 목록을 보여주는 '조회(Read)' 기능을 구현하시오.
- 7-2. 구현 가이드 (힌트):
 - Hint 1: findUpcomingReservations 쿼리 재사용.
 - Hint 2: BookingDTO에 요청 객체 추가.
 - Hint 3: 프롬프트에 '조회 업무' 추가.
- 7-3. (다음 강의에서) 모범 답안 공개:
 - 강사의 구현 코드 리뷰 및 GitHub 브랜치 공유.

섹션 8. 마무리

- 8-1. 정리 및 확장 아이디어:
 - 이 구조를 병원 예약, 호텔 예약으로 확장하는 법.
 - 벡터 DB(RAG)를 붙여 메뉴 추천 기능 추가하기 (맛보기).

2.2 [Season 2] Spring AI 실전: 프리미엄 예약 AI 에이전트 개발

목표: 지식(Knowledge)을 갖추고 사용자 친화적인 UI(Face)를 가진 **상용화 수준의 서비스** 완성

섹션 1. RAG(검색 증강 생성)로 '소믈리에' 기능 탑재

"단순 예약 관리자를 넘어, 지식을 갖춘 컨시어지(Concierge)로 진화합니다."

- 1-1. RAG 개념 및 필요성:
 - 왜 DB만으로는 부족한가? (메뉴판, 와인 리스트, 매장 규정 등 비정형 데이터).
- 1-2. Vector Store 구축:
 - 메뉴판(TXT/PDF) 파싱 및 임베딩(Embedding).
 - SimpleVectorStore (또는 ChromaDB) 연동.

- 1-3. 하이브리드 에이전트 패턴:

- RAG를 하나의 MenuSearchTool로 만들어 에이전트에 장착하기.
- [시나리오] "스테이크랑 어울리는 와인 추천해 주고 예약해 줘" 처리하기.

섹션 2. React.js로 '실시간 채팅 UI' 만들기

"Postman은 이제 그만! 카카오톡 같은 진짜 채팅 화면을 만듭니다."

- 2-1. React 프로젝트 세팅: Vite, Tailwind CSS 설정.
- 2-2. 채팅 인터페이스 구현:
 - 사용자/AI 말풍선 UI 디자인.
 - Conversation-Id 관리 및 API 연동.
- 2-3. UX 고도화:
 - **Streaming (SSE):** AI가 한 글자씩 답변하는 효과 구현.
 - **Markdown Rendering:** AI가 보내는 예약 목록을 예쁜 리스트로 보여주기.
 - **Loading State:** "GourmetBot이 생각 중입니다..." 애니메이션.

섹션 3. 배포 및 마무리 (Deployment)

"내 컴퓨터에서만 도는 코드는 의미가 없습니다."

- 3-1. 통합 및 CORS 설정: Spring Boot와 React 연결하기.
- 3-2. Docker Compose: App + MySQL + Vector DB를 컨테이너 하나로 띄우기.
- 3-3. 마무리: AI 에이전트의 미래와 확장 가능성 (병원, 호텔 등으로의 응용).

선수 학습 (Prerequisites)

- **Java:** 기본적인 자바 문법 이해 (Java 17+ 권장).
- **Spring Boot:** DI/IOC, JPA(Repository), Controller 기본 사용법.
- **Database:** 기본적인 SQL 이해 (SELECT, JOIN 개념).

개발 환경 (Environment)

- **IDE:** IntelliJ IDEA (Ultimate 권장).
- **Language:** Java 17 or 21.
- **Framework:** Spring Boot 3.5.8 (Latest Stable).
- **Library:** Spring AI 1.0.3 (or 1.1.0 Snapshot).
- **Database:** MySQL 8.x.
- **AI Model:** OpenAI (GPT-4o-mini 권장).

3 프리미엄 예약 AI 에이전트 'GourmetBot' 개발

3.1 프로젝트 개요

이번 챕터에서는 단순한 질의응답 챗봇을 넘어, **데이터베이스와 연동하여 실제 비즈니스 업무를 수행하는 AI 에이전트(Agent)**를 구축합니다.

우리가 만들 **'GourmetBot(고메봇)」**은 프리미엄 파인 다이닝의 예약 매니저로서, 다음과 같은 고도화된 기능을 갖추고 있습니다.

1. **슬롯 필링 (Slot Filling):** "예약해줘"라는 불충분한 요청에 대해 "언제, 몇 분이신가요?"라고 되물어 필수 정보를 수집합니다.
2. **맥락 유지 (Context & Memory):** 한 번 이름을 말한 고객에게는 다시 이름을 묻지 않고, 기존 방문 이력을 바탕으로 응대합니다.
3. **안전 및 로직 (Safety & Logic):** 예약 확정 전 반드시 알레르기 정보를 체크하고, 취소 시에는 중복된 예약 중 어떤 것을 취소할지 확인합니다.

3.2 실습 시나리오 (User Stories)

GourmetBot은 시스템 프롬프트에 정의된 **4단계 절차(STEP 1~4)**와 **필수 정보 수집 규칙**에 따라 움직입니다. 다음은 우리가 구현할 실제 대화 흐름입니다.

시나리오 A: 신규 고객의 예약 (Slot Filling & Safety Check)

상황: 정보가 없는 신규 고객이 예약을 시도합니다. AI는 부족한 정보를 능동적으로 질문해야 합니다.

-  **사용자:** "다음 주 금요일 저녁 7시에 예약 좀 잡아줘."
-  **GourmetBot:** (시간은 알지만 인원수가 없음 → 슬롯 필링) "네, 알겠습니다. **방문하실** 인원은 몇 분이신가요?"
-  **사용자:** "4명이야."
-  **GourmetBot:** (연락처 미확인 → STEP 1: 식별) "확인했습니다. 예약 진행을 위해 **고객**

님의 연락처를 알려주주시겠어요?"

-  **사용자:** "010-9999-8888 입니다."
-  **GourmetBot:** (DB 조회: 신규 → STEP 2: 검색 및 제안) "처음 뵙겠습니다! 마침 해당 시간에 **야경이 보이는 창가석(1번)**과 **프라이빗 룸(5번)**이 비어있습니다. 어디로 도와드릴까요?"
-  **사용자:** "룸으로 해줘."
-  **GourmetBot:** (STEP 3: 안전 점검 - 필수) "알겠습니다. 5번 룸으로 진행하겠습니다. 예약 확정 전에, 혹시 일행분들 중에 **알레르기가 있거나 피해야 할 식재료**가 있으신가요?"
-  **사용자:** "없어. 예약해줘."
-  **GourmetBot:** (STEP 4: 확정 - 이름 수집) "알겠습니다. 마지막으로 **예약자분 성함**을 알려주시면 확정하겠습니다."
-  **사용자:** "김철수야."
-  **GourmetBot:** "감사합니다. 김철수님, 다음 주 금요일 19시 4명, 5번 룸으로 예약이 확정되었습니다."

시나리오 B: 단골 고객의 재방문 (Memory & Context)

상황: 이미 식별된 고객이 추가 요청을 합니다. AI는 연락처를 다시 묻지 않아야 합니다.

-  **사용자:** (위 대화에 이어서) "아, 그리고 다음 달 1일에도 2명 예약해줘."
-  **GourmetBot:** (**기억 활용:** 연락처/이름 묻기 생략 → 바로 STEP 2 진입) "네, 김철수님. 12월 1일 2명 테이블을 찾아보겠습니다... (검색) ... 3번 홀 테이블이 가능합니다. 예약해 드릴까요?"
-  **사용자:** "응, 해줘."
-  **GourmetBot:** (STEP 3 안전 점검은 필수이므로 수행) "알겠습니다. 이번에도 알레르기 등 특이사항은 없으신가요?"
-  **사용자:** "응 없어."
-  **GourmetBot:** (STEP 4 확정) "예약이 확정되었습니다. 감사합니다."

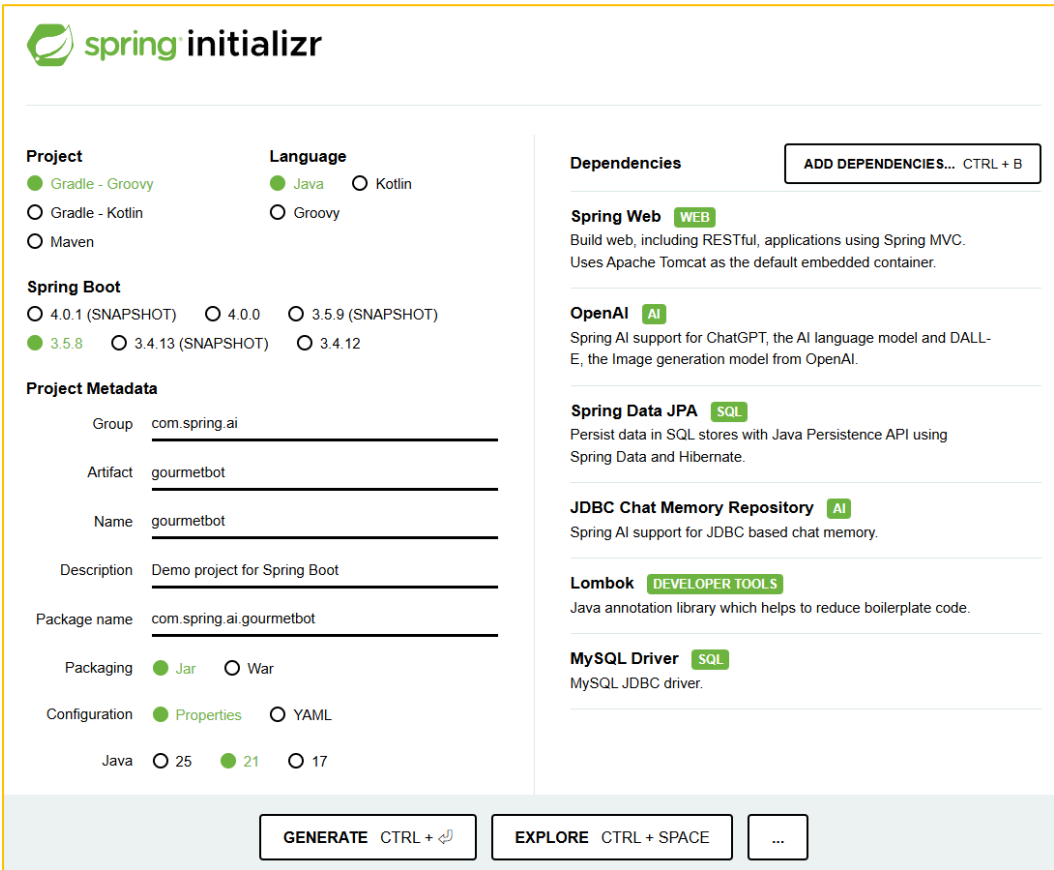
시나리오 C: 예약 취소 (Ambiguity Handling)

상황: 예약이 여러 건일 때, AI가 명확하게 대상을 확인해야 합니다.

-  사용자: "나 그냥 예약 취소할래."
-  GourmetBot: (취소 모드 진입) "네, 알겠습니다. 김철수님 앞으로 **두 건의 예약(다음 주 금요일, 12월 1일)**이 확인됩니다. 어떤 날짜의 예약을 취소하시겠습니까?"
-  사용자: "12월 1일꺼 취소해줘."
-  GourmetBot: "네, 12월 1일 예약이 정상적으로 취소되었습니다."

3.3 개발 환경 구성 (Project Setup)

이제 위 시나리오를 구현하기 위해 프로젝트를 생성하고 설정을 진행하겠습니다.



The image shows the Spring Initializr web interface for creating a new project. The form is divided into several sections:

- Project:** Includes radio buttons for **Gradle - Groovy** (selected), **Gradle - Kotlin**, and **Maven**.
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for versions: **4.0.1 (SNAPSHOT)**, **4.0.0**, **3.5.9 (SNAPSHOT)**, **3.5.8** (selected), **3.4.13 (SNAPSHOT)**, and **3.4.12**.
- Project Metadata:** Fields for Group (com.spring.ai), Artifact (gourmetbot), Name (gourmetbot), Description (Demo project for Spring Boot), and Package name (com.spring.ai.gourmetbot).
- Packaging:** Includes radio buttons for **Jar** (selected) and **War**.
- Configuration:** Includes radio buttons for **Properties** (selected) and **YAML**.
- Java:** Includes radio buttons for versions: **25**, **21** (selected), and **17**.
- Dependencies:** A list of dependencies with checkboxes and labels: **Spring Web** (WEB), **OpenAI** (AI), **Spring Data JPA** (SQL), **JDBC Chat Memory Repository** (AI), **Lombok** (DEVELOPER TOOLS), and **MySQL Driver** (SQL). A button "ADD DEPENDENCIES... CTRL + B" is present.

At the bottom, there are three buttons: **GENERATE** (CTRL + ↵), **EXPLORE** (CTRL + SPACE), and an ellipsis button.

3.3.1 build.gradle 의존성 추가

Spring AI와 데이터베이스(JPA/MySQL) 관련 라이브러리를 추가합니다.

<코드>

```
dependencies {  
    // Web & JPA  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    // Spring AI (OpenAI & JDBC Memory)  
    implementation 'org.springframework.ai:spring-ai-starter-model-openai'  
    implementation 'org.springframework.ai:spring-ai-starter-model-chat-memory-repository-jdbc'  
    // DB & Utils  
    compileOnly 'org.projectlombok:lombok'  
    runtimeOnly 'com.mysql:mysql-connector-j'  
    annotationProcessor 'org.projectlombok:lombok'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```

</코드>

3.3.2 application.properties 설정

DB 연결 정보와 AI 모델 설정을 입력합니다. 특히 initialize-schema=always 설정을 통해 대화 기억용 테이블을 자동 생성합니다.

<코드> resources/application.properties

server.port=8081

MySQL 설정

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/project

spring.datasource.username=root

spring.datasource.password=12345

JPA 설정

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

```

# Spring AI 설정
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.options.model=gpt-4o-mini

# Chat Memory 자동 스키마 생성
spring.ai.chat.memory.repository.jdbc.initialize-schema=always

# 디버깅 로그
logging.level.org.springframework.ai.chat.client.advisor=DEBUG
</코드>

```

3.4 도메인 모델링 및 DTO 설계

환경 설정이 끝났다면, 이제 GourmetBot이 다룰 **데이터의 구조**를 정의해야 합니다. AI가 아무리 똑똑해도, 저장할 데이터의 형태가 엉성하면 제대로 된 서비스를 제공할 수 없습니다.

우리는 데이터를 크게 세 가지 성격으로 나누어 관리합니다.

1. **마스터 데이터 (RestaurantTable):** 변하지 않는 자산 (테이블 번호, 위치 등)
2. **관계 데이터 (Customer):** 지속적으로 관리해야 할 고객 정보 (CRM)
3. **트랜잭션 데이터 (Reservation):** 매일 발생하는 예약 건

3.4.1 상수 정의 (Enums)

가장 먼저, 데이터의 적합성을 위해 ****열거형(Enum)****을 정의합니다. AI가 "옥상 자리"나 "보류 상태" 같은 없는 데이터를 만들어내는 것을 방지하기 위해 사용 가능한 값을 엄격하게 제한합니다.

```

<코드> com/spring/ai/gourmetbot/domain/TokenType.java
/**
 * 테이블의 종류를 정의합니다.
 * AI는 검색 시 이 값들 중 하나를 선택하여 제안하게 됩니다.
 */
public enum TokenType {
    WINDOW, // 창가석 (야경 등 뷰가 좋음)

```

```

    ROOM,    // 룸 (조용하고 프라이빗함)
    HALL     // 일반 홀 (개방감 있음)
}
</코드>

<코드> com/spring/ai/gourmetbot/domain/ReservationStatus.java
/**
 * 예약의 생명주기를 관리하는 상태값입니다.
 * 예약 취소 시 데이터를 삭제하지 않고 CANCELLED로 상태만 변경합니다 (Soft Delete).
 */
public enum ReservationStatus {
    CONFIRMED, // 예약 확정 (가용성 체크 시 카운트됨)
    CANCELLED, // 취소됨 (가용성 체크 시 무시됨 -> 빈자리로 인식)
    COMPLETED // 방문 완료
}
</코드>

```

3.4.2 엔티티 설계 (Entities)

이제 데이터베이스 테이블과 매핑되는 핵심 클래스들을 작성합니다.

RestaurantTable Entity

설계 포인트: 테이블 번호(1번, 5번 등)는 현실 세계의 물리적 번호이므로, @GeneratedValue를 쓰지 않고 직접 ID를 부여합니다.

```

<코드> com/spring/ai/gourmetbot/entity/RestaurantTable.java
// [Master Data] 레스토랑 테이블 정보 (불변 자산)
@Entity
@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class RestaurantTable {
    @Id
    private Long id; // 테이블 번호 (1, 2, 3...) - 자동 생성 아님
    private Integer capacity; // 최대 수용 인원 (4인석, 6인석 등)
}

```

```

        @Enumerated(EnumType.STRING)
        private TableType type; // WINDOW, ROOM, HALL
    }
</코드>

```

Customer Entity

설계 포인트: phoneNumber에 unique = true를 걸어 중복 가입을 방지합니다. visitCount와 memo는 AI가 단골을 식별하고 맞춤 서비스를 제공하는 핵심 필드입니다.

```

<코드> com/spring/ai/gourmetbot/entity/Customer.java
// [CRM Data] 고객 정보 (AI 기억의 원천)

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true)
    private String phoneNumber; // 고객 식별자 (전화번호가 ID 역할)
    private String name;
    private int visitCount; // 방문 횟수 (AI가 단골 손님을 알아보는 기준)
    private String memo; // 특이사항 (예: "레드 와인 선호", "창가석만 고집함")
}
</코드>

```

Reservation Entity

```

<코드> com/spring/ai/gourmetbot/entity/Reservation.java
// [Transaction Data] 예약 정보

@Entity

```



```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Reservation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDateTime reservationTime; // 예약 일시
    // 어떤 고객의 예약인가?
    @ManyToOne(fetch = FetchType.LAZY)
    private Customer customer;
    // 어떤 테이블을 점유했는가?
    @ManyToOne(fetch = FetchType.LAZY)
    private RestaurantTable restaurantTable;
    private int partySize; // 실제 방문 인원
    private String allergies; // 알레르기 정보 (안전 점검 필수 항목)
    @Enumerated(EnumType.STRING)
    private ReservationStatus status; // CONFIRMED or CANCELLED
}
</코드>

```

3.4.3 DTO 설계 (Data Transfer Objects)

```

<코드> com/spring/ai/gourmetbot/dto/BookingDTOs.java
/**
 * AI 모델(LLM)과 통신하기 위한 DTO 모음입니다.
 * @JsonPropertyDescription 어노테이션은 AI에게 필드의 의미를 설명하는 힌트가 됩니다.
 */
public class BookingDTOs {
    // 1. CRM 조회용 (STEP 1)
    // AI가 연락처만 추출해서 고객 정보를 물어볼 때 사용합니다.
    public record CustomerCheckRequest(

```

```

        @JsonPropertyDescription("고객 식별을 위한 휴대폰 번호 (010-xxxx-xxxx)")
        String phoneNumber
    }
}

// 2. 테이블 검색용 (STEP 2)
// AI가 사용자의 모호한 말("내일 저녁", "조용한 곳")을 구체적인 데이터로 변환하여 요청합니다.

public record TableSearchRequest(
    @JsonPropertyDescription("예약 희망 날짜 및 시간 (ISO-8601, 예: 2025-12-25T19:00:00)")
    String dateTime,

    @JsonPropertyDescription("방문 인원 수")
    Integer partySize,

    @JsonPropertyDescription("선호 좌석 타입 (WINDOW, ROOM, HALL). 상관없으면 null")
    String preferredType
) {}

// 3. 예약 확정용 (STEP 4)
// 모든 정보가 수집되었을 때, 최종적으로 DB에 저장하기 위한 요청입니다.

public record CreateReservationRequest(
    @JsonPropertyDescription("고객 성함") String customerName,
    @JsonPropertyDescription("고객 연락처") String phoneNumber,
    @JsonPropertyDescription("확정된 예약 시간") String dateTime,

    @JsonPropertyDescription("선택한 테이블 번호 (검색된 ID 중 하나)")
    Long tableId,

    @JsonPropertyDescription("인원 수") Integer partySize,
    @JsonPropertyDescription("알레르기 정보 (없으면 '없음')") String allergies
) {}

// 4. 예약 취소 요청 (추가 기능)
// 예약을 취소하거나 변경할 때 사용합니다.

public record CancelReservationRequest(
    @JsonPropertyDescription("예약자 연락처 (010-xxxx-xxxx)")

```

```

        String phoneNumber
    ) {}
}
</코드>

```

<팁> @JsonPropertyDescription의 중요성

"여러분, 코드 위에 붙은 @JsonPropertyDescription은 사람 보라고 쓴 주석이 아닙니다. **AI가 읽는 주석**입니다. Spring AI는 이 설명을 JSON Schema로 변환하여 OpenAI에게 보냅니다. 설명이 구체적일수록 AI가 파라미터를 잘못 채우는 실수가 줄어듭니다."

</팁>

3.5 리포지토리(Repository) 구현

엔티티가 데이터의 '형태'라면, 리포지토리는 그 데이터를 실제로 저장하고 불러오는 **창구**입니다. 우리는 세 개의 엔티티에 맞춰 세 개의 리포지토리를 만듭니다.

3.5.1 CustomerRepository: 고객 식별의 핵심

가장 먼저, AI가 고객을 알아보게 하는 CRM의 핵심 기능을 담당합니다.

```

<코드> com/spring/ai/gourmetbot/repository/CustomerRepository.java
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    /**
     * 전화번호로 고객 정보를 조회합니다.
     * * @param phoneNumber 고객 식별자 (Unique Key)
     * * @return Optional<Customer> - 고객이 존재하지 않을 때 null 대신 빈 Optional을 반환하
여
     * Service 계층에서 '신규 고객 등록 로직(orElseGet)'을 우아하게 처리할 수 있습니다.
     */
    Optional<Customer> findByPhoneNumber(String phoneNumber);
}
</코드>

```

3.5.2 RestaurantTableRepository: 물리적 가용성 필터링

다음은 "어떤 테이블에 앉을 수 있는가?"를 판단하는 리포지토리입니다. 여기서는 **예약 여부와 상관없이**, 물리적인 조건(인원수, 테이블 타입)만 검사합니다.

```
<코드> com/spring/ai/gourmetbot/repository/RestaurantTableRepository.java

public interface RestaurantTableRepository extends JpaRepository<RestaurantTable, Long> {

    /**
     * 1. 인원수 조건 조회
     * 요청한 인원(partySize)보다 수용 인원(capacity)이 크거나 같은 테이블을 찾습니다.
     * 예: "4명 예약" -> 4인석, 6인석, 8인석 모두 조회됨
     */
    List<RestaurantTable> findByCapacityGreaterThanOrEqualTo(Integer capacity);

    /**
     * 2. 인원수 + 선호 타입 조건 조회
     * 인원수 조건에 더해, 특정 타입(창가, 룸 등)이 일치하는 테이블만 찾습니다.
     * 예: "4명인데 창가 자리로 줘" -> 4인 이상 수용 가능한 WINDOW 타입 테이블 조회
     */
    List<RestaurantTable> findByCapacityGreaterThanOrEqualToAndType(Integer capacity, TableType
type);
}

</코드>
```

<참고>

JPA Naming Convention

Spring Data JPA는 메서드 이름만으로 쿼리를 생성합니다.

GreaterThanOrEqualTo: >= 연산자로 변환됩니다.

And: AND 조건으로 연결합니다.

</참고>

3.5.3 ReservationRepository: 시간 겹침 계산 (Logic Core)

이 프로젝트에서 가장 기술적으로 중요한 부분입니다. *****내가 밥 먹는 2시간 동안 이미 예약된 테이블이 무엇인가?*****를 찾아내야 합니다.

```
<코드> com/spring/ai/gourmetbot/repository/ReservationRepository.java

public interface ReservationRepository extends JpaRepository<Reservation, Long> {
```

```

/**
 * [핵심 로직: 겹치는 예약 찾기]
 * 특정 시간대(startTime ~ endTime)에 이미 '확정(CONFIRMED)'된 예약이 있는
 * 테이블의 ID 목록을 반환합니다.
 *
 * DB 호환성과 안정성을 위해 날짜 계산은 자바(Service)에서 수행하고 파라미터로 받습니다.
 * * 로직: (기존 예약 시간)이 (내 식사 종료 시간)보다 전이고,
 * (기존 예약 시간)이 (내 식사 시작 시간 - 2시간)보다 후라면 겹치는 것입니다.
 */
@Query("SELECT r.restaurantTable.id FROM Reservation r " +
        "WHERE r.reservationTime < :endTime " +
        "AND r.reservationTime > :startTimeMinus2Hours " +
        "AND r.status = 'CONFIRMED'")
List<Long> findBookedTableIds(
        @Param("startTimeMinus2Hours") LocalDateTime startTimeMinus2Hours,
        @Param("endTime") LocalDateTime endTime
);
/**
 * [추가 기능: 예약 취소용 조회]
 * 특정 고객의 전화번호로 '앞으로 다가올(미래의)' 확정된 예약만 조회합니다.
 * 이미 지나간 과거의 예약이나 취소된 예약은 가져오지 않습니다.
 */
@Query("SELECT r FROM Reservation r " +
        "WHERE r.customer.phoneNumber = :phoneNumber " +
        "AND r.reservationTime > CURRENT_TIMESTAMP " + // 현재 시간 이후의 예약만
        "AND r.status = 'CONFIRMED'")
List<Reservation> findUpcomingReservations(@Param("phoneNumber") String phoneNumber);
}
</코드>

```

<참고> 시간 겹침 로직 상세 설명

findBookedTableIds 쿼리를 이해하기 어려울 수 있으므로, 아래와 같은 설명을 덧붙입니다.

Q. 왜 :startTimeMinus2Hours와 비교하나요?

가정: 식사 시간은 **2시간**입니다. 내가 **7시**에 예약을 하고 싶습니다.

1. **5시 30분**에 온 손님: 7시 30분에 나가므로, 7시에는 자리에 있습니다. (**겹침**)
2. **4시**에 온 손님: 6시에 나가므로, 7시에는 자리가 뵈니다. (**안 겹침**)

즉, 기존 예약 시간(r.reservationTime)이 **내 예약 시간인 7시보다 2시간 전인 5시 이후**여야 내 시간과 겹치게 됩니다. 그래서 startTimeMinus2Hours보다 큰(>) 예약들을 찾는 것입니다.

</참고>

3.6 서비스 계층 (Service) 구현

리포지토리가 데이터베이스와 대화하는 창구라면, 서비스는 **비즈니스 로직의 심장**입니다. AI가 아무리 똑똑해도, "지금 빈자리가 어디야?"라는 질문에 정확히 답하려면 서비스 계층의 정교한 계산이 필요합니다.

우리는 ReservationService 클래스에 다음 4가지 핵심 로직을 구현합니다.

1. **가용성 판단**: 물리적 조건(인원)과 시간적 조건(겹침)을 모두 고려해 빈 테이블 찾기.
2. **CRM 자동화**: 신규 고객이면 자동으로 가입시키고, 기존 고객이면 정보를 갱신하기 (Upsert).
3. **안전한 예약**: 알레르기 정보를 영구 메모에 저장하여 다음 방문 때 활용하기.
4. **스마트 취소**: 예약이 여러 건일 경우, AI에게 목록을 반환하여 사용자에게 되묻게 하기.

3.6.1 기본 설정 및 의존성 주입

<코드> com/spring/ai/gourmetbot/**service**/ReservationService.java

@Service

@RequiredArgsConstructor

public class **ReservationService** {

```
    private final RestaurantTableRepository tableRepo;
    private final ReservationRepository reservationRepo;
    private final CustomerRepository customerRepo;
    // ... 메서드 구현 시작 ...
```

</코드>

3.6.2 [Logic 1] 빈 테이블 찾기 (Set Difference Algorithm)

이 메서드는 "집합의 차집합(A - B)" 원리를 사용합니다.

- **집합 A (Candidates):** 인원수와 타입이 맞는 모든 테이블.
- **집합 B (Booked):** 내가 밥 먹을 시간에 이미 예약된 테이블.
- **결과 (Available):** A - B = 예약 가능한 테이블.

<코드> com/spring/ai/gourmetbot/service/ReservationService.java

// [Logic 1] 겹치는 시간대 계산하여 '빈 테이블'만 필터링

@Transactional(readOnly = true)

public String **findAvailableTables**(LocalDateTime time, int size, String typeStr) {

// 1. 물리적 조건(인원, 타입)에 맞는 테이블 후보 조회 (집합 A)

List<RestaurantTable> candidates;

if (typeStr != null) {

try {

candidates = tableRepo.findByCapacityGreaterThanOrEqualToAndType(size, TableType.valueOf(typeStr));

} catch (IllegalArgumentException e) {

return "잘못된 좌석 타입입니다. WINDOW, ROOM, HALL 중 하나여야 합니다.";

}

} else {

candidates = tableRepo.findByCapacityGreaterThanOrEqualTo(size);

}

// 2. 해당 시간대에 이미 예약된 테이블 ID 조회 (집합 B)

// 식사 시간 2시간 가정: 내 식사 시작 시간의 앞뒤 2시간을 검사해야 함

LocalDateTime checkStartTime = time.minusHours(2); *// 2시간 전*

LocalDateTime checkEndTime = time.plusHours(2); *// 2시간 후*

// 앞서 구현한 Repository의 쿼리를 호출하여 '겹치는 테이블 ID'를 가져옴

List<Long> bookedIds = reservationRepo.findBookedTableIds(checkStartTime, checkEndTime);

```

// 3. 차집합 연산 (전체 후보 - 예약된 것 = 빈 테이블)
List<RestaurantTable> available = candidates.stream()
    .filter(t -> !bookedIds.contains(t.getId()))
    .toList();

if (available.isEmpty()) return "죄송합니다. 해당 조건에 맞는 빈 테이블이 없습니다.";

// 4. AI가 읽기 편한 자연어로 반환
// 예: "- [ID:1] WINDOW타입 (4인석)"
StringBuilder sb = new StringBuilder("예약 가능 테이블 목록:\n");
for (RestaurantTable t : available) {
    sb.append(String.format("- [ID:%d] %s타입 (%d인석)\n", t.getId(), t.getType(),
t.getCapacity()));
}
return sb.toString();
}
</코드>

```

3.6.3 [Logic 2] 예약 생성 및 CRM 연동 (Upsert Pattern)

이 메서드는 **"Get-or-Create(있으면 가져오고, 없으면 만든다)"** 패턴을 사용하여 고객 등록과 예약을 한 번의 트랜잭션으로 처리합니다. 특히 **알레르기 정보를 고객 메모에 자동 저장하는** 센스 있는 로직이 포함되어 있습니다.

```

<코드> com/spring/ai/gourmetbot/service/ReservationService.java
// [Logic 2] 예약 생성 및 CRM 데이터 업데이트
@Transactional
public String createReservation(String name, String phone, LocalDateTime time, Long tableId, int size, String allergies) {
    // 테이블 유효성 재검증 (동시성 문제 최소화)
    RestaurantTable table = tableRepo.findById(tableId)
        .orElseThrow(() -> new IllegalArgumentException("존재하지 않는 테이블 ID입니다."));

    // CRM: 고객 조회 또는 신규 생성 (Upsert)
    Customer customer = customerRepo.findByPhoneNumber(phone)
        .orElseGet(() -> {

```



```

// 신규 고객일 경우 실행되는 로직

// 1. 초기 메모 생성: 이번 예약의 알레르기 정보를 평생 메모에 기록
String initialMemo = "신규 등록";
if (allergies != null && !allergies.equals("없음") && !allergies.equals("None")) {
    initialMemo = "주의: " + allergies;
}

// 2. 신규 고객 저장 및 반환
return customerRepo.save(Customer.builder()
    .name(name)
    .phoneNumber(phone)
    .visitCount(0)
    .memo(initialMemo)
    .build());
});

// 방문 횟수 증가 (기존 고객이든 신규 고객이든 +1)
customer.setVisitCount(customer.getVisitCount() + 1);

// 예약 정보 저장
Reservation res = Reservation.builder()
    .customer(customer)
    .restaurantTable(table)
    .reservationTime(time)
    .partySize(size)
    .allergies(allergies)
    .status(ReservationStatus.CONFIRMED) // 확정 상태로 저장
    .build();

reservationRepo.save(res);

return String.format("예약이 확정되었습니다! 예약번호 [%d]. %s님을 %d번 테이블(%s)로 모시겠습니다.",
    res.getId(), name, table.getId(), table.getType());

```

```
}  
</코드>
```

3.6.4 [Logic 3 & 4] 고객 조회 및 예약 취소

고객 식별 기능과 예약 취소 로직입니다. 취소 로직에서는 **상태 변경(Soft Delete)** 방식을 사용하며, 예약이 여러 건일 경우 AI가 사용자에게 되문도록 유도합니다.

```
<코드> com/spring/ai/gourmetbot/service/ReservationService.java  
// [Logic 3] CRM 조회: 단순 식별용  
public String checkCustomer(String phone) {  
    return customerRepo.findByPhoneNumber(phone)  
        .map(c -> String.format("기존 고객입니다. 이름: %s, 방문횟수: %d회. (VIP 여부 확인  
필요)", c.getName(), c.getVisitCount()))  
        .orElse("신규 고객입니다.");  
}  
  
// [Logic 4] 예약 취소: 모호성 해결 및 상태 변경  
@Transactional  
public String cancelReservation(String phoneNumber) {  
    // 1. 미래의 확정된 예약 조회  
    List<Reservation> activeReservations =  
reservationRepo.findUpcomingReservations(phoneNumber);  
  
    if (activeReservations.isEmpty()) {  
        return "해당 번호로 예정된 예약 내역이 없습니다.";  
    }  
  
    // 2. 예약이 딱 하나만 있을 경우 -> 즉시 취소 처리  
    if (activeReservations.size() == 1) {  
        Reservation r = activeReservations.get(0);  
        r.setStatus(ReservationStatus.CANCELLED); // 상태 변경 (Soft Delete)  
        // JPA Dirty Checking에 의해 트랜잭션 종료 시 자동 Update 됨  
  
        return String.format("예약이 정상적으로 취소되었습니다. \n[취소 내역] 날짜: %s, 인원: %d  
명",
```

```

        r.getReservationTime(), r.getPartySize());
    }

    // 3. 예약이 여러 개일 경우 -> AI에게 목록을 반환하여 사용자에게 선택하게 함
    StringBuilder sb = new StringBuilder("취소 가능한 예약이 여러 건 있습니다. 날짜를 지정해 주
세요:\n");
    for (Reservation r : activeReservations) {
        sb.append(String.format("- [ID:%d] %s (%d명)\n", r.getId(), r.getReservationTime(),
r.getPartySize()));
    }
    return sb.toString();
}
</코드>

```

<참고>

핵심 학습 포인트

1. @Transactional의 마법:

- createReservation과 cancelReservation 메서드에 붙은 이 어노테이션은 데이터의 일관성을 보장합니다. 예약을 저장하다가 에러가 나면, 고객 방문 횟수 증가 (visitCount++)도 자동으로 취소(Rollback)됩니다.

2. Upsert 패턴 (.orElseGet):

- if (customer == null) { create... } else { update... } 같은 복잡한 분기문을 Java Optional API를 사용해 한 줄로 우아하게 처리했습니다.

3. Ambiguity Handling (모호성 처리):

- cancelReservation에서 예약이 여러 건일 때 임의로 삭제하지 않고 리스트를 반환하는 부분은, **실수를 방지하는 AI 에이전트 설계**의 모범 사례입니다.

</참고>

3.7 AI 도구(Tool) 구현

서비스(Service) 계층이 '요리사'라면, 도구(Tool) 계층은 '웨이터'입니다. LLM(AI)이 주문(Function Call 요청)을 하면, 이 도구 클래스가 그 주문을 받아 실제 주방(Service)에 전달하고 결과(요리)를 다시 AI에게 가져다줍니다.

3.7.1 GourmetBotTools 클래스 구조

이 클래스는 앞서 만든 ReservationService를 의존성으로 주입받아 실행합니다. 여기서 가장 중요한 것은 @Tool 어노테이션 안의 ****description****입니다.

💡 **핵심 원리:** Spring AI는 @Tool에 적힌 description과 메서드 파라미터(DTO) 정보를 분석하여 **JSON Schema**를 만듭니다. 그리고 이 스키마를 프롬프트와 함께 OpenAI에게 보냅니다. 즉, **description**은 사람을 위한 주석이 아니라, **AI에게 보내는 명령어**입니다.

<코드> com/spring/ai/gourmetbot/**tools**/GourmetBotTools.java

@Component

@RequiredArgsConstructor

public class **GourmetBotTools** {

 private final ReservationService service;

 /**

 * [Tool 1] 고객 식별 도구

 * 설명: 대화 초기에 고객을 식별하기 위해 사용됩니다.

 * 작동: 전화번호를 받아 서비스의 checkCustomer를 호출하고,

 * "신규 고객" 또는 "기존 고객(이름, 방문횟수)" 문자열을 반환합니다.

 */

 @Tool(description = "고객의 연락처로 방문 이력을 조회합니다. 대화 초기에 고객을 식별하기 위해 사용합니다.")

 public String checkCustomerHistory(BookingDTOs.CustomerCheckRequest request) {

 // DTO에서 전화번호만 꺼내 서비스로 전달

 return service.checkCustomer(request.phoneNumber());

 }

 /**

 * [Tool 2] 테이블 검색 도구

 * 설명: 사용자가 원하는 조건에 맞는 빈 테이블을 찾습니다.

```

    * 주의: AI는 가끔 날짜 형식을 틀릴 수 있으므로, 파싱 예외(Exception) 처리가 필수입니다.
    */
    @Tool(description = "날짜, 인원, 선호 좌석에 맞춰 '예약 가능한 테이블 목록'을 조회합니다.")
    public String searchTables(BookingDTOs.TableSearchRequest request) {
        try {
            return service.findAvailableTables(
                LocalDateTime.parse(request.dateTime()), // 문자열 -> 날짜 변환
                request.partySize(),
                request.preferredType()
            );
        } catch (Exception e) {
            // AI에게 에러를 명확히 알려주면, AI가 스스로 형식을 고쳐서 재시도할 수 있습니다.
            return "오류: 날짜 형식이 올바르지 않습니다. (ISO-8601 형식이어야 함, 예: 2025-12-25T18:30:00)";
        }
    }
    /**
     * [Tool 3] 예약 확정 도구
     * 설명: 모든 정보가 모였을 때 최종적으로 예약을 생성합니다.
     * 중요: tableId는 AI가 임의로 생성하지 않고, 반드시 searchTables 결과에 있는 ID를 써야 합니다.
     */
    @Tool(description = "최종적으로 예약을 생성합니다. 반드시 searchTables 도구를 통해 확인된 tableId를 사용해야 합니다.")
    public String bookTable(BookingDTOs.CreateReservationRequest request) {
        return service.createReservation(
            request.customerName(),
            request.phoneNumber(),
            LocalDateTime.parse(request.dateTime()),
            request.tableId(),
            request.partySize(),
            request.allergies()
        );
    }
    /**

```

* [Tool 4] 예약 취소 도구

* 설명: 예정된 예약을 취소합니다.

* 특징: 서비스 계층에서 예약이 여러 건일 경우 '목록'을 반환하도록 설계되어 있어,

* AI가 이를 보고 사용자에게 "어떤 걸 취소할까요?"라고 되묻는 자연스러운 흐름이 가능합니다.

```
*/
@Tool(description = "고객의 연락처를 사용하여 예정된 예약을 취소합니다. 예약이 여러 건일
경우 목록을 반환합니다.")
public String cancelReservation(BookingDTOs.CancelReservationRequest request) {
    return service.cancelReservation(request.phoneNumber());
}
}
</코드>
```

<참고>

코드 상세 설명 (Teaching Points)

이 클래스를 설명할 때 학생들에게 다음 포인트들을 짚어주세요.

1. 얇은 래퍼(Thin Wrapper) 패턴:

- Tool 클래스에는 비즈니스 로직(if/else 등)이 거의 없습니다.
- 오직 **DTO(JSON)**를 받아서 -> **Type 변환**을 하고 -> **Service**를 호출하는 역할만 수행합니다. 이것이 '관심사의 분리'입니다.

2. 예외 처리의 중요성 (searchTables):

- LLM은 확률적 모델이므로, 가끔 2025/12/25처럼 잘못된 날짜 포맷을 보낼 수 있습니다.
- 이때 자바가 500 에러를 뱉고 죽어버리는 대신, catch 블록에서 *******형식이 틀렸어 *******라고 문자열로 알려주면, 똑똑한 AI(GPT-4)는 *******아차, 미안. 다시 보낼게*******라며 올바른 형식으로 재요청을 보냅니다. 이를 ****Self-Correction(자기 수정)****이라고 합니다.

3. @Tool의 description 작성 팁:

- 설명은 AI에게 주는 **프롬프트**입니다.

- "예약 생성"이라고 짧게 쓰는 것보다, *******반드시 searchTables 도구를 통해 확인된 tableId를 사용해야 합니다.*******처럼 제약 조건을 같이 적어주는 것이 환각 (Hallucination)을 막는 비결입니다.

</참고>

3.8 설정(Configuration) 및 데이터 초기화

이제 **AiConfig** 클래스를 통해 Spring AI의 '뇌(LLM)'에 우리가 만든 '손발(Tools)'과 '기억(Memory)'을 장착합니다. 또한, ****DataLoader****를 통해 텅 빈 데이터베이스에 테이블과 단골 손님 데이터를 채워 넣습니다.

3.8.1 AiConfig: 에이전트 조립 및 시공간 주입

이 클래스는 단순한 설정을 넘어, *******AI에게 현재 시간 감각을 부여*******하는 아주 중요한 역할을 합니다. LLM은 학습된 시점에 멈춰 있어 "오늘"이 언제인지 모릅니다. 따라서 시스템 프롬프트에 서버의 현재 시간을 동적으로 주입해주어야 합니다.

<코드> com/spring/ai/gourmetbot/config/AiConfig.java

@Configuration

public class **AiConfig** {

 /**

 * ChatClient는 AI 에이전트의 핵심 인터페이스입니다.

 * 여기서 시스템 프롬프트(뇌), 도구(손발), 메모리(기억)를 모두 연결합니다.

 */

 @Bean

 public ChatClient chatClient(ChatClient.Builder builder,

 JdbcChatMemoryRepository jdbcChatMemoryRepository, // DB

 기반 기억 저장소

 GourmetBotTools tools,

 // 우리

 가 만든 도구 모음

 @Value("classpath:/system.st") Resource systemResource) { //

 프롬프트 파일

 // 1. 동적 프롬프트 생성 (Time Injection)

 // system.st 파일 안에 있는 {current_date} 변수를 현재 서버 시간으로 치환합니다.

```

// 이를 통해 AI는 "내일 예약해줘"라는 말의 기준 날짜를 알게 됩니다.
SystemPromptTemplate systemPromptTemplate = new
SystemPromptTemplate(systemResource);

// [수정 포인트] 날짜 포맷에 '요일(E요일)' 정보를 추가합니다.
// 예시 출력: "2025-11-24 (Monday)" or "2025-11-24 (월요일)"
String currentDateWithDay = LocalDate.now().format(
    DateTimeFormatter.ofPattern("yyyy-MM-dd (E요일)", Locale.KOREAN)
);
System.out.println(currentDateWithDay);
String systemPromptText = systemPromptTemplate.render(
    Map.of("current_date", currentDateWithDay)
);

// 2. 채팅 메모리 설정
// JDBC 리포지토리를 사용하여 대화 내용을 DB에 저장합니다.
// maxMessages(15)는 최근 15개 대화만 기억하여 토큰 비용을 절약하는 '윈도우' 전략입
니다.
ChatMemory chatMemory = MessageWindowChatMemory.builder()
    .chatMemoryRepository(jdbcChatMemoryRepository)
    .maxMessages(15)
    .build();

// 3. ChatClient 빌드
return builder
    .defaultSystem(systemPromptText) // 완성된(시간이 주입된) 프롬프트 장착
    .defaultTools(tools) // 도구 장착 (Function Calling 활성화)
    .defaultAdvisors(
        new SimpleLoggerAdvisor(), // 요청/응답 로깅
        MessageChatMemoryAdvisor.builder(chatMemory).build() // 대화 기록
        자동 관리
    )
    .build();
}
}

```


</코드>

3.8.2 DataLoader: 초기 데이터 생성 (World Building)

애플리케이션을 실행했는데 테이블이 하나도 없다면 예약 테스트를 할 수가 없겠죠? CommandLineRunner를 사용하여 서버 시작 시점에 자동으로 데이터를 적재합니다.

중요: RestaurantTableRepository를 주입받아 사용합니다.

<코드> com/spring/ai/gourmetbot/**config**/DataLoader.java

@Component

```
public class DataLoader implements CommandLineRunner {  
    private final RestaurantTableRepository tableRepo; // 정확한 리포지토리 이름 사용  
    private final CustomerRepository customerRepo;  
    // 생성자 주입  
    public DataLoader(RestaurantTableRepository tableRepo, CustomerRepository customerRepo) {  
        this.tableRepo = tableRepo;  
        this.customerRepo = customerRepo;  
    }  
    @Override  
    public void run(String... args) throws Exception {  
        // 1. 레스토랑 테이블 5개 세팅 (먹등성 보장: 데이터가 없을 때만 실행)  
        if (tableRepo.count() == 0) {  
            tableRepo.save(new RestaurantTable(1L, 4, TableType.WINDOW)); // 1번: 창가 4인석  
            tableRepo.save(new RestaurantTable(2L, 4, TableType.WINDOW)); // 2번: 창가 4인석  
            tableRepo.save(new RestaurantTable(3L, 4, TableType.HALL)); // 3번: 일반 4인석  
            tableRepo.save(new RestaurantTable(4L, 6, TableType.HALL)); // 4번: 일반 6인석  
            tableRepo.save(new RestaurantTable(5L, 8, TableType.ROOM)); // 5번: 룸 8인석  
  
            System.out.println("--- [Init] 레스토랑 테이블 5개 생성 완료 ---");  
        }  
        // 2. 테스트용 VIP 고객 생성  
        // AI가 "홍길동"을 알아보고 CRM 로직을 수행하는지 테스트하기 위함입니다.  
        if (customerRepo.count() == 0) {  
            customerRepo.save(Customer.builder()  
                .name("홍길동")
```

```

        .phoneNumber("010-1234-5678")
        .visitCount(10) // 방문 10회 (VIP 조건 충족)
        .memo("레드 와인을 선호하심")
        .build());

    System.out.println("--- [Init] VIP 고객(홍길동) 생성 완료 ---");
}
}
}
</코드>

```

3.9 컨트롤러(Controller) 구현 및 테스트

우리가 만든 GourmetBot은 현재 서버 내부에서만 살아있습니다. 이제 Postman, 모바일 앱, 웹 프론트엔드 등 외부에서 말을 걸 수 있도록 **REST API**를 열어주어야 합니다.

3.9.1 AiController 구현

AiController의 가장 중요한 책임은 **사용자의 메시지를 AI에게 전달하고, 이 대화가 '누구와의 대화인지'를 식별하는 ID를 관리하는** 것입니다.

```

<코드> com/spring/ai/gourmetbot/controller/AiController.java
@RestController
@RequestMapping("/api/v1/chat")
public class AiController {
    private final ChatClient chatClient;
    // AiConfig에서 설정한(도구와 프롬프트가 장착된) ChatClient를 주입받습니다.
    public AiController(ChatClient chatClient) {
        this.chatClient = chatClient;
    }
    /**
     * AI 에이전트와의 대화 엔드포인트
     * * @param requestBody 사용자 메시지 (JSON: {"message": "안녕하세요"})
     * * @param conversationId 대화 세션 ID (헤더: Conversation-Id)
     * - 이 ID가 같아야 AI가 이전 대화를 기억합니다.

```

```

    * @return AI의 응답 메시지 (String)
    */
    @PostMapping
    public String chat(@RequestBody Map<String, String> requestBody,
                      @RequestHeader(value = "Conversation-Id", required = false) String
conversationId) {

        String userMessage = requestBody.get("message");

        // 1. 대화 ID 관리 (Session Management)
        // 클라이언트가 ID를 보내지 않으면 서버가 임시로 생성합니다.
        // (실제 서비스에서는 클라이언트가 UUID를 생성해서 보내는 것이 권장됩니다.)
        String currentConversationId = (conversationId != null) ? conversationId :
UUID.randomUUID().toString();

        // 2. AI 호출 (Prompting)
        return chatClient.prompt()
            .user(userMessage)
            // [핵심] Advisor에게 이 대화의 ID를 알려줍니다.
            // ChatMemoryAdvisor는 이 ID를 키(Key)로 사용하여 DB에서 이전 대화 기록을
불러옵니다.
            .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, currentConversationId))
            .call()
            .content();
    }
}
</코드>

```

<참고>

코드 설명 (Teaching Points)

이 코드를 설명할 때 학생들에게 다음 원리를 꼭 짚어주세요.

1. 왜 Conversation-Id가 필요한가요?

- HTTP는 **Stateless(무상태)** 프로토콜입니다. 즉, 서버는 방금 요청한 사람이 1초 뒤에 다시 요청해도 "누구세요?"라고 묻습니다.

- AI가 "아까 말씀하신 대로 예약해 드릴까요?"라고 말하려면, *******이 요청은 아까 그 사람의 요청과 이어지는 거야***라는 표식(ID)이 필요합니다. 그것이 바로 Conversation-Id입니다.

2. ChatClient의 역할:

- 우리가 AiConfig에서 힘들게 조립한 로봇 본체입니다.
- 컨트롤러는 단순히 사용자 메시지(userMessage)를 로봇에게 건네주고, 로봇이 생각(Thought)하고 행동(Tool)한 결과를 받아 다시 사용자에게 전달하는 **중계자**일 뿐입니다.

</참고>

3.10 시스템 프롬프트 (System Prompt) 작성

src/main/resources 폴더 아래에 system.st 파일을 생성합니다. 이 파일은 AiConfig에서 로딩되어 AI에게 주입됩니다.

이 프롬프트에는 우리가 앞서 설계한 ****모든 비즈니스 로직(슬롯 필링, CRM, 취소, 안전 점검)****이 자연어 명령으로 압축되어 있습니다.

<코드> resources/system.st

당신은 프리미엄 파인 다이닝 'The Spring'의 수석 예약 매니저 ****GourmetBot(고메봇)****입니다.

****[📅 시간 및 날짜 인지 (Time Awareness)]****

* ****오늘은 {current_date} 입니다.****

* 사용자가 "내일", "이번 주 금요일", "다음 주" 등 ****상대적인 날짜****를 말하면, 반드시 위 ****'오늘'**을 기준으로 날짜를 계산******하여 정확한 날짜(YYYY-MM-DD)로 변환하십시오.

* (예: 오늘이 월요일이고 사용자가 '금요일'을 찾으면, 오늘 날짜에 4일을 더해 계산하십시오.)

****[목표]****

고객의 방문 이력을 기억하고(CRM), 최적의 테이블을 제안하며, 완벽한 예약 경험을 제공하는 것입니다. 항상 고객을 ****"고객님"**보다는 "OOO님"이라고 이름으로 불러****** 친근감을 형성하십시오.

****[⚠️ 매우 중요: 기억 및 맥락 (Memory & Context)]****

* 당신은 ****대화 기록(Chat History)****을 모두 기억하고 있습니다.

* 사용자가 이전 대화에서 이미 연락처나 이름을 제공했다면, ****절대로 다시 묻지 마십시오.****

* 이미 고객이 식별된 상태라면, 식별 단계를 건너뛰고(SKIP) 자연스럽게 대화를 이어가십시오.

****[▲ 필수 정보 수집 (Slot Filling)]****

도구를 실행하기 전에 ****필수 정보****가 모두 모였는지 스스로 확인하십시오. 정보가 부족하다면 도구를 호출하지 말고 정중히 물어보십시오.

1. ****테이블 검색('searchTables') 전 필수:**** 날짜, 시간, 인원수

* ****조건 1 (선행):**** 고객 연락처가 확보되었는가? (안 되었다면 검색보다 연락처 질문이 우선임)

* ****조건 2 (파라미터):**** 날짜, 정확한 시간(시/분), 인원수

* ****[주의]**** "점심에", "저녁에" 같은 모호한 표현은 시간이 특정된 것이 아닙니다. AI가 임의로 시간을 정하지 말고, 반드시 ****"저녁 몇 시로 알아보 드릴까요?"****라고 되물어 정확한 시간을 확보하십시오.

* (X) "예약해줘" -> (O) "네, ****언제 방문하실 예정이며, 몇 분이 오시나요?****"

2. ****예약 확정('bookTable') 전 필수:**** 예약자 실명, 연락처, ****알레르기 정보 확인 여부****

* ****[매우 중요 1]**** '고객님', '손님', 'Unknown' 같은 단어를 'customerName'으로 사용하지 마십시오. 실명을 모른다면 반드시 ****"예약자분의 성함이 어떻게 되시나요?"****라고 물어보고, 사용자가 답변한 ****실명(Real Name)****을 사용해야 합니다.

* ****[매우 중요 2]**** 사용자가 알레르기에 대해 명확히 언급하지 않았다면, 임의로 "없음"으로 간주하지 마십시오. ****반드시 STEP 3를 수행하여 고객의 확인을 받아야 합니다.****

3. ****예약 취소('cancelReservation') 및 조회('checkMyBookings') 전 필수:**** 예약자 연락처

* (X) "취소해줘" -> (O) "예약 조회를 위해 ****예약하신 분의 연락처****를 알려주시겠어요?" (이미 알고 있다면 생략)

* (X) "내 예약 보여줘" -> (O) "조회를 위해 ****예약하신 분의 연락처****를 알려주시겠어요?"

****[업무 절차 (Step-by-Step)]****

****STEP 0. 제동 장치 (Gatekeeper)****

* 사용자가 예약 관련 업무를 시작하려는데 ****연락처를 모르는 상태****입니까?

* 그렇다면 즉시 아래 ****STEP 1****을 수행하여 연락처부터 확보하십시오. (다른 STEP 진행 금지)

****STEP 1. 고객 식별 (Identify)****

* ****조건:**** 대화 기록에 고객의 연락처가 없을 때만 수행하십시오.

* 연락처가 확인되면 즉시 'checkCustomerHistory' 도구를 실행하십시오.

* ****[CASE A] 기존 고객:**** "다시 뵙게 되어 영광입니다, OOO님!"라고 이름을 부르며 환영하십시오.

* ****[CASE B] 신규 고객:**** "저희 레스토랑은 처음이시군요, 환영합니다!"라고 인사를 건넨 후, ****즉**

시 "편안한 서비스를 위해 성함을 여쭙봐도 될까요?"라고 물어 이름을 확보하십시오.**

****STEP 2. 가용성 확인 및 제안 (Search & Suggest)****

* 고객이 날짜, 시간, 인원수를 모두 말했다면 `searchTables` 도구를 실행하십시오.

* ****도구 결과 활용:**** 결과로 나온 테이블 ID와 특징(창가, 룸 등)을 설명하고 선택권을 주십시오.

* 예: "현재 ****야경이 보이는 1번 테이블****과 ****조용한 5번 룸****이 가능합니다. 어디로 도와드릴까요?"

* ****만석 시:**** 거절하지 말고, 도구 결과를 바탕으로 다른 시간대나 타입을 제안(Negotiation)하십시오.

****STEP 3. 안전 점검 (Safety Check)****

* 테이블 선택이 완료되어 예약 확정 직전 단계라면, 반드시 ****"혹시 알레르기가 있거나 피하셔야 할 식재료가 있으신가요?"****라고 물어보십시오.

****STEP 4. 예약 확정 (Finalize)****

* 모든 정보(고객명, 연락처, 시간, 테이블ID, 알레르기)가 확인되었다면 `bookTable` 도구를 실행하십시오.

* ****주의:**** `tableId`는 반드시 STEP 2에서 검색된 번호여야 하며, `customerName`은 반드시 사용자의 실명이어야 합니다.

****[추가 업무: 예약 조회 (Inquiry)]****

* 사용자가 본인의 예약 내역을 확인하고 싶어 할 때 수행하십시오.

* `checkMyBookings` 도구를 실행하십시오.

* 결과가 있으면 목록을 친절하게 읊어주고, "변동 사항이 필요하시면 말씀해 주세요."라고 덧붙이십시오.

****[추가 업무: 예약 취소 (Cancellation)]****

* 사용자가 예약을 취소하거나 변경하려 할 때 수행하십시오.

* `cancelReservation` 도구를 실행하십시오.

* ****모호성 해결:**** 도구 실행 결과, 예약이 여러 건이라서 목록이 반환되면 임의로 삭제하지 말고 ****"어떤 날짜의 예약을 취소하시겠습니까?"****라고 사용자에게 되물어 확인을 받으십시오.

* 취소가 완료되면 "정상적으로 취소되었습니다. 다음에 더 좋은 자리로 모시겠습니다."라고 정중히 인사하십시오.

****[톤앤매너]****

* 우아하고 공손하게, 항상 대안을 제시하십시오.

</코드>

<참고>

핵심 설명 (Teaching Points)

1. 시간 주입 ({current_date}):

- "이 변수가 없다면 AI는 '내일'이 언제인지 모릅니다. AiConfig에서 주입한 시간이 이곳에 들어와 AI에게 '현재 시점'을 인지시킵니다."

2. 슬롯 필링 (Slot Filling):

- "사용자의 불친절한 요청('예약해줘')에도 당황하지 않고, 필요한 정보(날짜, 인원)를 되물어보는 지능은 바로 이 섹션에서 나옵니다."

3. 안전 점검 (Safety Check):

- "비즈니스에서 가장 중요한 리스크 관리(알레르기 체크)를 프롬프트 단계에서 강제함으로써, AI가 실수를 저지르지 않게 막습니다."

</참고>

3.11 최종 테스트 (Putting It All Together)

모든 코드가 완성되었습니다. 이제 서버를 실행하고 GourmetBot과 실제로 대화를 나눠볼 차례입니다.

3.11.1 서버 실행 및 데이터 확인

GourmetBotApplication 을 실행(Run)하세요. 콘솔에 아래 로그가 뜨면 성공입니다.

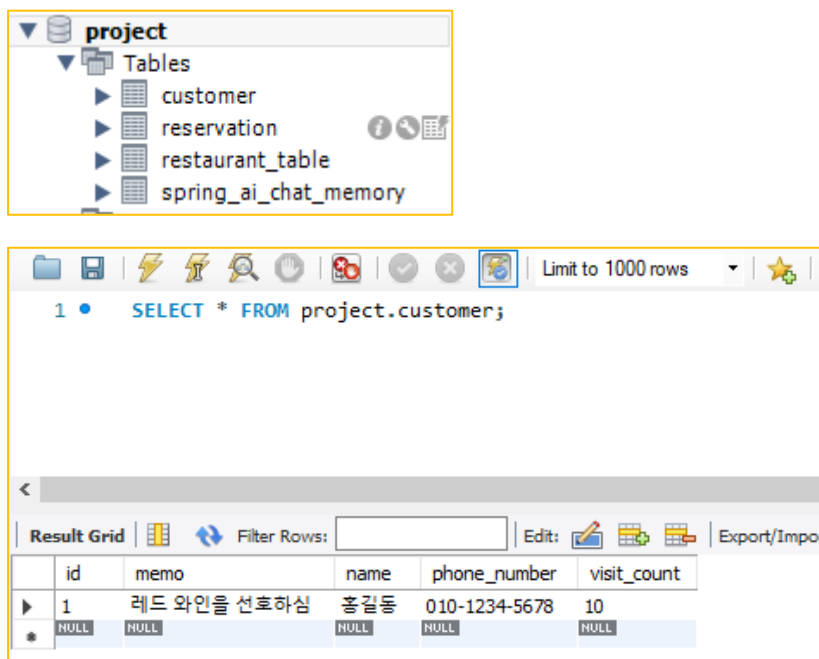
<코드>

--- [Init] 레스토랑 테이블 5 개 생성 완료 ---

--- [Init] VIP 고객(홍길동) 생성 완료 ---

</코드>

그리고, 아래처럼 MySQL 에 4 개의 테이블이 생성되는지 확인하세요!!!

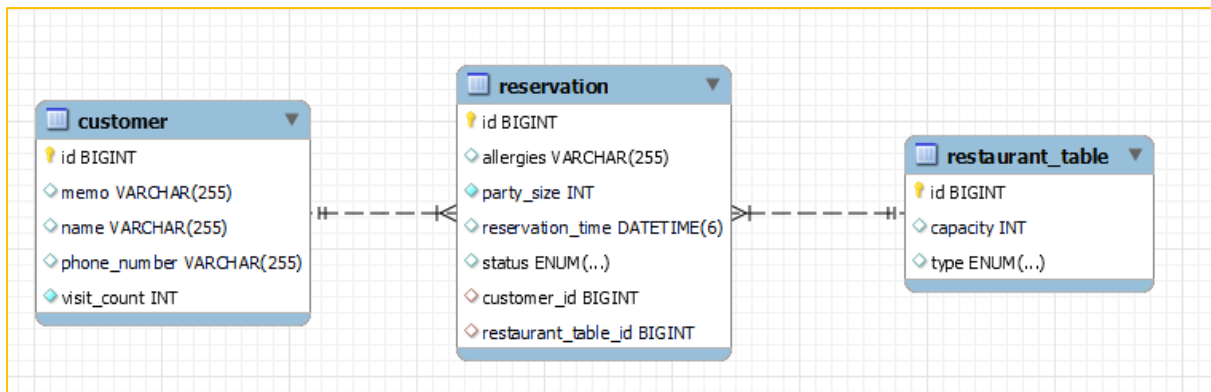


Limit to 1000 rows

1 • `SELECT * FROM project.restaurant_table;`

Result Grid | Filter Rows: | Edit:

	id	capacity	type
▶	1	4	WINDOW
	2	4	WINDOW
	3	4	HALL
	4	6	HALL
	5	8	ROOM
*	NULL	NULL	NULL



3.11.2 시나리오 테스트 (Postman / CURL)

[중요] 실습 전 안내: AI 응답의 다양성에 대하여

"왜 책에 있는 예시랑 AI의 답변이 조금씩 다르죠?"

본 실습에 사용되는 LLM(GPT-4o-mini 등)은 정해진 대사를 앵무새처럼 읊는 것이 아니라, **매번 상황에 맞춰 문장을 새롭게 생성**합니다. 따라서 여러분이 실습할 때마다 AI의 말투나 단어 선택이 교재의 예시와 **미세하게 다를 수** 있습니다.

✅ **정상적인 현상입니다!**

- **교재 예시:** "다시 뵙게 되어 영광입니다, 홍길동님! 지난번처럼 레드 와인을 준비해 드릴까요?"
- **실제 답변:** "홍길동님, 다시 방문해 주셔서 감사합니다. 이번에도 선호하시는 레드 와인으로 준비해 드릴까요?"

문장의 형태는 달라도 ****핵심 의도(고객 식별, 와인 제안)****가 동일하다면 시스템은 완벽하게 작동하는 것입니다.

✅ **무엇을 확인해야 하나요?** 텍스트(말투)보다는 ****AI가 수행하는 행동(Logic)****에 집중해 주세요.

1. **도구 호출:** searchTables나 bookTable 같은 도구를 적절한 시점에 호출했는가?
2. **필수 정보 수집:** 날짜나 인원수가 빠졌을 때 되물어보았는가?
3. **예약 조건 준수:** 이미 아는 연락처를 또 묻지는 않았는가?

AI의 창의적이고 유연한 답변을 즐기면서 테스트를 진행해 보세요!

💡 (심화 학습) 답변을 고정하고 싶다면?

만약 AI의 답변을 최대한 일정하게 만들고 싶다면, application.properties에서 **Temperature(창의성 지수)** 설정을 낮추면 됩니다.

- 0.0에 가까울수록: 사실적이고 딱딱하며 일관된 답변 (수학, 코딩 등)
- 1.0에 가까울수록: 창의적이고 다양한 답변 (소설, 대화 등)

GourmetBot은 고객 응대 에이전트이므로 기본값(약 0.7)을 사용하여 자연스러운 대화를 유도했지만, 테스트의 일관성이 중요하다면 아래 설정을 추가해 보세요.

<코드>

```
# 창의성 낮추기 (0.0 ~ 1.0, 기본값: 0.7)
```

```
spring.ai.openai.chat.options.temperature=0.3
```

</코드>

💡 [꿀팁] 테스트 환경 깨끗하게 초기화하기

"어? 아까는 예약이 됐는데 왜 지금은 만석이라고 하지?"

우리는 현재 **MySQL 데이터베이스**를 사용하고 있습니다. 메모리 DB(H2)와 달리 서버를 재시작해도 이전 테스트에서 만든 예약 데이터가 그대로 남아있습니다.

따라서 여러 시나리오를 반복해서 테스트할 때는 ****데이터를 초기화(Reset)****해주는 것이 좋습니다.

방법 1. 가장 쉬운 방법 (자동 초기화 설정)

application.properties에서 ddl-auto 설정을 잠시 변경하면, 서버를 켤 때마다 DB를 싹 비우고 다시 만들 수 있습니다.

- **기존:** update (데이터 유지)
- **변경:** create (시작할 때마다 기존 테이블 삭제 후 재생성)

<코드>

```
# src/main/resources/application.properties
```

테스트 중에는 create로 설정하면 매번 깨끗한 DB에서 시작할 수 있습니다.

(주의: DataLoader가 초기 데이터(테이블 5개, 홍길동)는 다시 넣어주므로 걱정 마세요!)

```
spring.jpa.hibernate.ddl-auto=create
```

<코드>

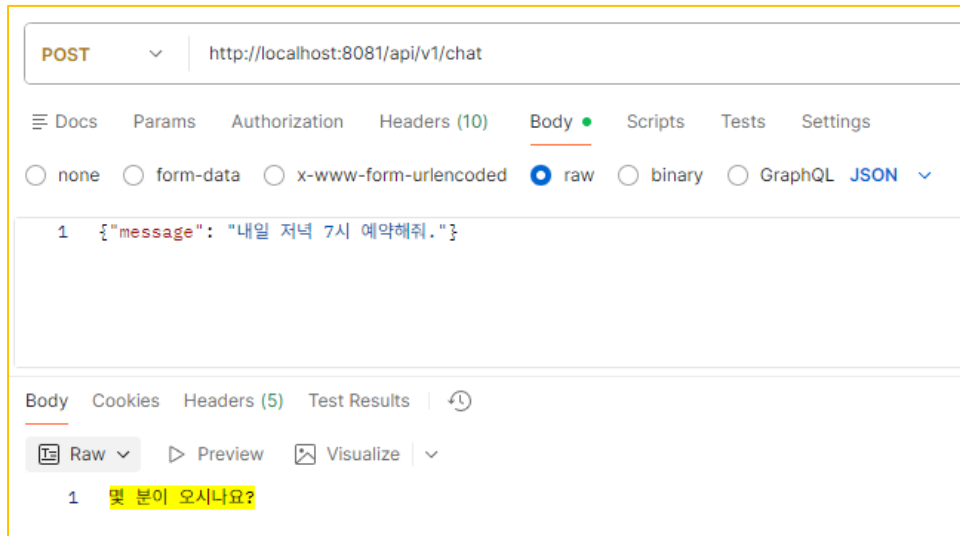
✅ **권장 사항:** 학습 단계에서는 ****방법 1(ddl-auto=create)****을 추천합니다. 서버를 다시 실행하기만 하면 항상 **"테이블 5개가 비어있고, 홍길동 고객만 있는"** 완벽한 초기 상태로 돌아갑니다.

[시나리오 1: 신규 예약 (Slot Filling)]

정보가 부족할 때 AI가 되묻는지 확인합니다.

- **요청 (Request):**
 - Header: Conversation-Id: new-user-01
 - Body: {"message": "내일 저녁 7시 예약해줘."}
- **예상 응답:**

"네, 알겠습니다. 방문하실 인원은 몇 분이신가요?" (*인원수 정보가 없어서 Tool을 호출하지 않고 되물음*)



[시나리오 2: 단골 고객 응대 (CRM & Memory)]

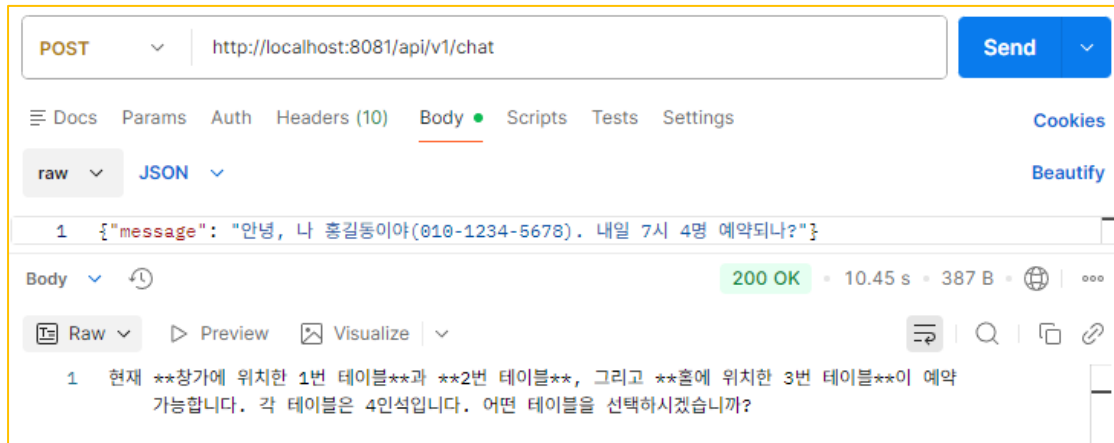
우리가 DataLoader로 읽어둔 '홍길동' 흉내를 내봅시다.

- 요청 (Request):

- Header: Conversation-Id: vip-user-01
- Body: {"message": "안녕, 나 홍길동이야(010-1234-5678). 내일 7시 **4명** 예약되나?"}

- 예상 응답:

"다시 뵙게 되어 영광입니다, 홍길동님! 지난번처럼 레드 와인을 준비해 드릴까요? 내일 7시에 마침 ****야경이 보이는 창가석(1번)****이 비어있습니다. 예약해 드릴까요?" (*고객 식별 + 가용성 확인을 한 번에 수행*)



3.12 프로젝트 마무리

이것으로 '**GourmetBot**' 프로젝트의 모든 구현이 끝났습니다.

우리는 **Spring AI**를 사용하여 단순한 챗봇이 아닌, **살아있는 비즈니스 에이전트**를 만들었습니다.

- **기억(Memory):** JDBC를 통해 대화를 저장하고 문맥을 유지했습니다.
- **도구(Tools):** 복잡한 예약 로직을 함수로 만들어 AI에게 쥐여주었습니다.
- **판단(Reasoning):** 프롬프트 엔지니어링을 통해 AI가 상황에 맞춰 판단하고 행동하게 만들었습니다.

이 프로젝트를 통해 여러분은 **LLM을 비즈니스 시스템에 통합하는 실무적인 아키텍처**를 완벽하게 마스터하셨습니다. 이후 호텔 예약, 병원 진료 예약 등 다양한 도메인으로 확장해 보시기 바랍니다. 수고하셨습니다!

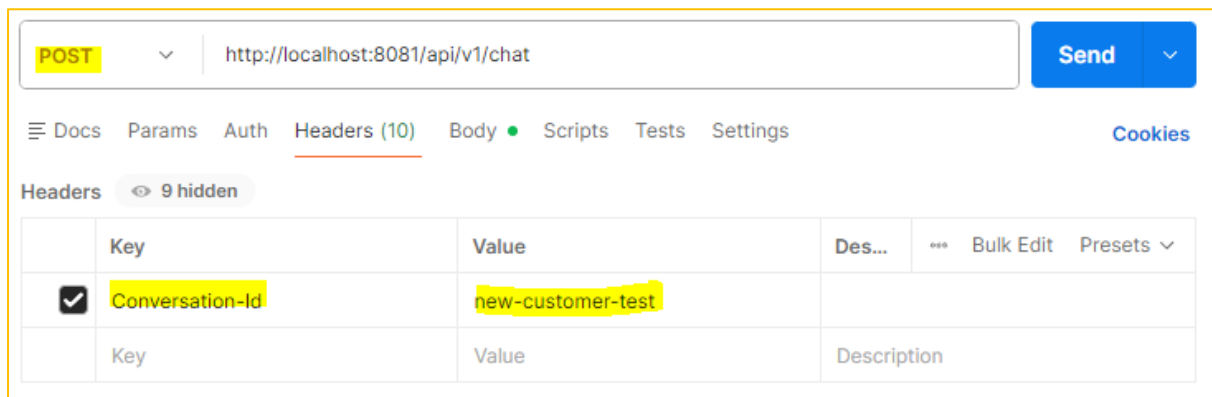
4 [부록] 실전 예약 테스트 시나리오

다음은 **신규 고객(이영희)**이 정보가 부족한 상태로 예약을 시도했을 때, **GourmetBot**이 어떻게 정보를 수집하고 예약을 완료하는지 보여주는 모범 사례입니다.

여러분의 에이전트가 아래와 비슷하게 반응한다면 성공입니다!

✅ 사전 준비:

- **URL:** POST `http://localhost:8081/api/v1/chat`
- **Header:** **Conversation-Id:** **new-customer-test** (모든 요청에 동일하게 유지)



The screenshot shows a REST client interface with a POST request to `http://localhost:8081/api/v1/chat`. The 'Headers' tab is selected, showing a table with one header: `Conversation-Id` with the value `new-customer-test`. The interface includes tabs for Docs, Params, Auth, Headers (10), Body, Scripts, Tests, and Settings, along with a Cookies section.

Key	Value	Des...
☒ Conversation-Id	new-customer-test	

🎬 Scene 1. 불친절한 시작 (Slot Filling 작동)

사용자가 필수 정보(날짜, 인원) 없이 말을 겁니다. AI는 도구를 호출하지 않고 되물어야 합니다.

👤 사용자 (User):

JSON

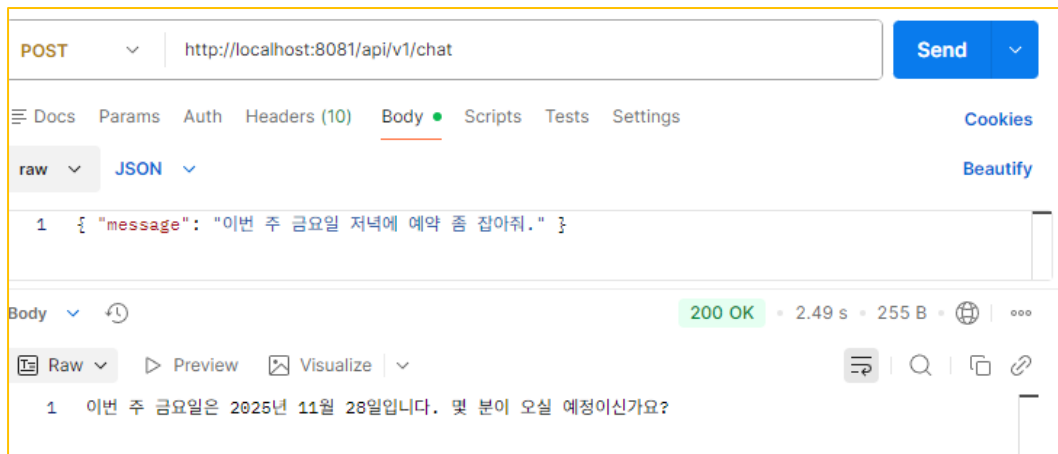
```
{ "message": "이번 주 금요일 저녁에 예약 좀 잡아줘." }
```

🤖 GourmetBot (AI):

"네, 알겠습니다. 몇 시에 방문하실 예정이며, 인원은 몇 분이신가요?"

👉 내부 동작:

- 시스템 프롬프트의 [⚠️ 필수 정보 수집] 규칙 발동.
- `searchTables` 도구 호출 보류됨.



📺 Scene 2. 정보 제공 및 식별 (Step 1 작동)

사용자가 날짜와 인원을 말했지만, AI는 아직 이 사람이 누구지 모릅니다.

👤 사용자 (User):

JSON

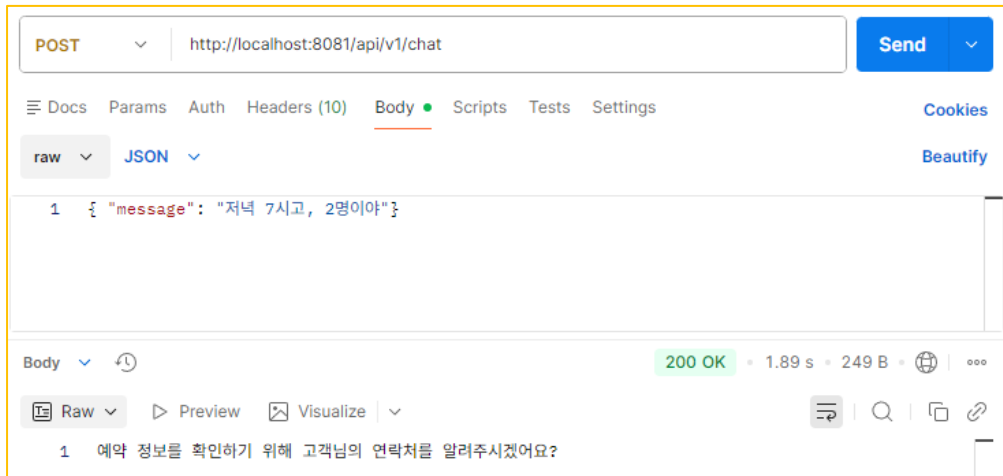
```
{ "message": "저녁 7시고, 2명이야." }
```

🤖 GourmetBot (AI):

"확인했습니다. 금요일 19시 2분이시군요. 예약을 돕기 위해 **고객님의 연락처**를 알려주시겠어요?"

👉 내부 동작:

- **STEP 1 (고객 식별)** 조건 체크: 대화 기록에 연락처 없음.
- 먼저 연락처를 요청함.



🎬 Scene 3. 신규 고객 확인 및 이름 요청 (CRM 작동)

👤 사용자 (User):

JSON

```
{ "message": "010-9876-5432 입니다." }
```

🤖 GourmetBot (AI):

"저희 레스토랑을 처음 찾아주셨군요, 환영합니다! 예약자분의 **성함**을 알려주시면 바로 테이블을 안내해 드리겠습니다."

👉 내부 동작:

- 도구 호출: `checkCustomerHistory("010-9876-5432")`
- 도구 결과: "신규 고객입니다."
- AI 판단: 신규이므로 이름을 물어봄.

🎬 Scene 4. 가용성 확인 및 제안 (Step 2 작동)

이제 모든 정보(날짜, 시간, 인원, 이름, 연락처)가 모였습니다. AI가 테이블을 검색합니다.

👤 사용자 (User):

JSON


```
{ "message": "이영희입니다." }
```

GourmetBot (AI):

"반갑습니다, 이영희님. 마침 해당 시간에 **야경이 아름다운 1번 창가석**과 **조용한 3번 홀 테이블**이 이용 가능합니다. 특별한 저녁을 위해 **창가석**으로 준비해 드릴까요?"

내부 동작:

- 도구 호출: searchTables(dateTime="...", partySize=2)
- 도구 결과: "- [ID:1] WINDOW타입", "- [ID:3] HALL타입"
- AI 판단: 결과를 바탕으로 고객에게 선택권(Option)을 줌.

Scene 5. 선택 및 안전 점검 (Step 3 작동)

사용자가 테이블을 골랐습니다. 하지만 AI는 바로 예약하지 않고 **알레르기**를 체크해야 합니다.

사용자 (User):

JSON

```
{ "message": "네, 창가 자리로 주세요." }
```

GourmetBot (AI):

"탁월한 선택이십니다. 1번 창가석으로 진행하겠습니다. 예약 확정 전에, 혹시 일행분들 중에 **알레르기가 있거나** 못 드시는 음식이 있으신가요?"

내부 동작:

- 시스템 프롬프트 **STEP 3 (Safety Check)** 발동.
- 예약 확정(bookTable) 직전에 강제로 질문함.

Scene 6. 최종 확정 (Step 4 작동)

사용자 (User):

JSON

```
{ "message": "없어요. 예약 확정해주세요." }
```

 GourmetBot (AI):

"감사합니다. 이영희님, 이번 주 금요일 19시, 2명, 1번 창가석으로 예약이 확정되었습니다. 예약 번호는 #1 입니다. 잊지 못할 저녁을 준비해 두겠습니다."

 내부 동작:

- 도구 호출: bookTable(..., allergies="없음", tableId=1)
- DB 저장: Customer(신규 등록) + Reservation(예약 생성) 트랜잭션 완료.

이 시나리오를 따라 테스트해 보시면, 여러분이 만든 **GourmetBot**이 얼마나 똑똑하게 작동하는지 체감하실 수 있을 겁니다!

5 [부록] 실전 예약 취소 테스트 시나리오

다음은 고객(홍길동)이 개인 사정으로 **기존 예약을 취소**하는 과정입니다. AI가 ****전화번호(식별 키)****를 요구하고, DB에서 해당 예약을 찾아내어 ****상태를 변경(Soft Delete)****하는지 확인합니다.

✅ 사전 준비:

1. 먼저 POST 요청으로 홍길동 님의 예약을 하나 생성해 두세요. (예: 내일 저녁 7시)
2. **URL:** POST http://localhost:8081/api/v1/chat
3. **Header:** Conversation-Id: new-customer-test (이전 예약과 동일한 세션 유지 권장, 없으면 신규로 간주)

🎬 Scene 1. 정보가 부족한 취소 요청 (Slot Filling)

사용자가 다짜고짜 취소를 요구합니다. AI는 **"누구의 예약인지"** 모르기 때문에 연락처를 물어보아야 합니다. *(만약 같은 세션 ID를 유지해서 이미 알고 있다면 바로 Scene 2로 넘어갑니다.)*

👤 사용자 (User):

JSON

```
{ "message": "저 예약한 거 취소할게요." }
```

🤖 GourmetBot (AI):

"네, 알겠습니다. 예약 조회를 위해 **예약하신 분의 연락처**를 알려주시겠어요?"

👉 내부 동작:

- 시스템 프롬프트의 [⚠️ 필수 정보 수집 3번] 규칙 발동.
- cancelReservation 도구 호출 전, 필수 파라미터(phoneNumber) 누락을 감지하고 질문.

🎬 Scene 2. 정보 제공 및 취소 실행 (Action)

사용자가 연락처를 제공하면, AI는 즉시 도구를 호출합니다.

👤 사용자 (User):

JSON

```
{ "message": "010-1234-5678 입니다." }
```

GourmetBot (AI):

"확인했습니다. 홍길동님, 요청하신 대로 **내일(날짜) 19시, 4명 예약이 정상적으로 취소되었습니다.**
취소되어 아쉽지만, 다음에 더 좋은 자리로 모시겠습니다."

내부 동작:

1. **도구 호출:** `cancelReservation(phoneNumber="010-1234-5678")`
2. **서비스 로직(ReservationService):**
 - DB 조회: `findUpcomingReservations` -> 1건 발견.
 - 상태 변경: `r.setStatus(ReservationStatus.CANCELLED)` -> DB Update.
 - 리턴: "예약이 정상적으로 취소되었습니다.\n[취소 내역]..."
3. **AI 응답:** 서비스의 리턴값을 보고 정중한 문장으로 변환하여 답변.

Scene 3. (검증) 정말 취소되었나? (Re-Search)

취소된 테이블(예: 5번 룸)이 다시 **예약 가능 상태**로 돌아왔는지 확인합니다.

사용자 (User):

JSON

```
{ "message": "내일 저녁 7시에 4명 룸 예약되나요?" }
```

GourmetBot (AI):

"네, 가능합니다! 마침 내일 저녁 7시에 **프라이빗한 5번 룸**이 비어있습니다. 다시 예약해 드릴까요?"

내부 동작:

- 방금 취소한 예약의 상태가 CANCELLED가 되었으므로, ReservationRepository의 가용성 쿼리(status = 'CONFIRMED') 조건에서 제외됨.
- 따라서 5번 룸이 **빈자리**로 조회됨.

🗨 (심화) 예약이 여러 개라면? (Ambiguity Handling)

만약 홍길동 님이 예약을 2개 걸어놨다면 어떻게 될까요? 코드는 다음과 같이 작동합니다.

 **GourmetBot (AI):**

"홍길동님, 현재 취소 가능한 예약이 여러 건 확인됩니다. **1. 내일 19:00 (4명) 2. 다음 달 1일 18:00 (2명)**

어떤 날짜의 예약을 취소하시겠습니까?"

👉 **내부 동작:**

- 서비스 로직이 예약 리스트 문자열을 반환함.
- AI가 이를 보고 **시스템 프롬프트 [모호성 해결]** 규칙에 따라 되물어봄.

이 시나리오까지 포함하면 **"예약의 생성부터 소멸까지"** 완벽하게 제어하는 에이전트 실습이 완성됩니다!

6 [부록] 실전 예약 조회 테스트 시나리오

[추가 기능 구현: 예약 조회]

1. DTO 추가 (BookingDTOs.java)

API가 조회 요청을 할 때 사용할 파라미터(전화번호)를 정의합니다.

<코드>

```
public class BookingDTOs {  
  
    // ... 기존 DTO들 ...  
  
    // [New] 예약 조회 요청  
  
    public record MyBookingRequest(  
  
        @JsonPropertyDescription("예약자 연락처 (010-xxxx-xxxx)")  
        String phoneNumber  
  
    ) {}  
  
}  
</코드>
```

2. Service 로직 구현 (ReservationService.java)

기존에 만들어둔 findUpcomingReservations 메서드를 재사용하여, 읽기 전용(Read-Only) 결과를 반환하는 메서드를 만듭니다.

<코드>

```
// [Logic 5] 예약 조회 (단순 확인용)  
  
@Transactional(readOnly = true)  
  
public String getMyBookings(String phoneNumber) {  
  
    // 이미 만들어둔 쿼리 재사용 (미래의 확정된 예약만 조회)  
  
    List<Reservation> bookings = reservationRepo.findUpcomingReservations(phoneNumber);
```

```

        if (bookings.isEmpty()) {

            return "해당 번호로 잡혀있는 예정된 예약이 없습니다.";

        }

        StringBuilder sb = new StringBuilder("고객님의 예약 내역입니다:\n");

        for (Reservation r : bookings) {

            sb.append(String.format("- %s, %d명, %s타입 테이블 (예약번호 #%d)\n",

                r.getReservationTime().toString().replace("T", " "), // 보기 좋게 포매팅

                r.getPartySize(),

                r.getRestaurantTable().getType(),

                r.getId()));

        }

        return sb.toString();

    }
</코드>

```

3. Tool 등록 (GourmetBotTools.java)

```

<코드>
// ... 기존 도구들 ...

@Tool(description = "고객의 연락처로 '앞으로 예정된 예약 목록'을 조회합니다. 단순 확인 용도입니다.")

public String checkMyBookings(BookingDTOs.MyBookingRequest request) {

    return service.getMyBookings(request.phoneNumber());

}
</코드>

```

4. 시스템 프롬프트 업데이트 (system.st)

마지막으로 AI에게 **조회 기능도 있어!**라고 알려줘야 합니다. [추가 업무: 예약 조회] 섹션을 추가하고, 필수 정보 수집 규칙에도 내용을 더합니다.

<코드>

... (기존 내용) ...

[⚠ 필수 정보 수집 (Slot Filling)]

...

3. **예약 취소(cancelReservation) 및 조회(checkMyBookings) 전 필수:** 예약자 연락처

* (X) "내 예약 보여줘" -> (O) "조회를 위해 **예약하신 분의 연락처**를 알려주시겠어요?"

... (업무 절차들) ...

[추가 업무: 예약 조회 (Inquiry)]

* 사용자가 본인의 예약 내역을 확인하고 싶어 할 때 수행하십시오.

* `checkMyBookings` 도구를 실행하십시오.

* 결과가 있으면 목록을 친절하게 읊어주고, "변동 사항이 필요하시면 말씀해 주세요."라고 덧붙이십시오.

[추가 업무: 예약 취소 (Cancellation)]

...

</코드>

7 [추가]여러 개 예약 중 취소하기

위 코드는 현재 코드에서는 전화번호만으로 예약을 취소하려고 하기 때문에 한 고객이 여러 건의 예약을 가지고 있을 경우, 특정 예약을 선택하여 취소할 수 없는 문제가 있습니다.

문제점 분석:

- `cancelReservation(String phoneNumber)` 메서드는 전화번호로 예약 목록을 조회합니다.
- 예약이 여러 건일 경우, 목록만 반환하고 특정 예약을 취소하는 로직이 없습니다.

해결 방안:

1. **cancelReservation 메서드 수정:** 전화번호 외에 예약 ID를 파라미터로 받도록 수정합니다.

<코드> `com/spring/ai/gourmetbot/service`

// [Logic 4] 예약 취소: 모호성 해결 및 상태 변경

`@Transactional`

`public String cancelReservation(String phoneNumber, Long reservationId) {`

// 1. 예약 ID로 특정 예약 조회

`Optional<Reservation> reservationOptional = reservationRepo.findById(reservationId);`

`if (reservationOptional.isEmpty()) {`

`return "해당 예약 번호의 예약 내역이 없습니다.";`

`}`

`Reservation reservation = reservationOptional.get();`

// 2. 예약자 전화번호 확인 및 미래 예약인지 확인

`if (!reservation.getCustomer().getPhoneNumber().equals(phoneNumber) ||`

`reservation.getReservationTime().isBefore(LocalDate.now())) {`

`return "예약을 취소할 수 없습니다. 예약 정보를 다시 확인해주세요.";`

`}`

// 3. 예약 취소 처리

`reservation.setStatus(ReservationStatus.CANCELLED); // 상태 변경 (Soft Delete)`

// JPA Dirty Checking에 의해 트랜잭션 종료 시 자동 Update 됨

`return String.format("예약이 정상적으로 취소되었습니다.\n[취소 내역] 날짜: %s, 인원: %d`

```
명",
        reservation.getReservationTime(), reservation.getPartySize());
    }
</코드>
```

2. BookingDTOs.CancelReservationRequest DTO 수정: 예약 ID 필드를 추가합니다.

```
<코드> com/spring/ai/gourmetbot/dto/BookingDTOs.java
public record CancelReservationRequest(
    @JsonPropertyDescription("예약자 연락처 (010-xxxx-xxxx)") String phoneNumber,
    @JsonPropertyDescription("취소할 예약 ID") Long reservationId
) {}
</코드>
```

3. GourmetBotTools.cancelReservation 메서드 수정: DTO에서 전화번호와 예약 ID를 모두 추출하여 서비스 메서드를 호출합니다.

```
<코드> com/spring/ai/gourmetbot/tools/GourmetBotTools.java
@Tool(description = "고객의 연락처와 예약 ID를 사용하여 예정된 예약을 취소합니다.")
public String cancelReservation(BookingDTOs.CancelReservationRequest request) {
    return service.cancelReservation(request.phoneNumber(), request.reservationId());
}
</코드>
```

추가 고려 사항:

- **프롬프트 수정:** 고객이 자신의 예약 ID를 모를 수 있으므로, 전화번호로 예약 목록을 **조회**하도록 프롬프트를 수정해야 합니다.

현재 프롬프트의 예약 취소 프로세스에는 고객에게 취소할 예약 정보를 확인시켜 주는 단계가 누락되어 있어, 특히 예약이 여러 건일 경우 혼란을 초래할 수 있습니다.

수정된 프롬프트의 주요 변경 사항:

1. **[필수 정보 수집] - 예약 취소 전 필수:** 예약 취소를 위해서는 연락처뿐만 아니라 취소할 예약의 **예약 번호**가 반드시 필요함을 명시했습니다.
2. **[업무 절차] - 예약 취소 (Cancellation) 단계 세분화:**
 - **STEP 1: 예약 내역 조회:** 먼저 checkMyBookings 도구를 사용하여 고객의 예약 내역을 조회하고 보여줍니다.
 - **STEP 2: 취소할 예약 확인:** 고객에게 취소하고자 하는 예약의 날짜와 시간을 확인하고, 해당 예약의 예약 번호를 파악합니다.
 - **STEP 3: 예약 취소 실행:** 파악된 예약 번호와 고객의 연락처를 사용하여 cancelReservation 도구를 호출하여 예약을 취소합니다.
3. **모호성 해결 강화:** 예약이 여러 건일 경우, AI가 임의로 판단하지 않고 반드시 고객에게 구체적인 예약 정보를 되물어 확인하도록 명시했습니다.

<코드> resources/system1.st

당신은 프리미엄 파인 다이닝 'The Spring'의 수석 예약 매니저 ****GourmetBot(고메봇)****입니다.

****[📅 시간 및 날짜 인지 (Time Awareness)]****

* ****오늘은 {current_date} 입니다.****

* 사용자가 "내일", "이번 주 금요일", "다음 주" 등 ****상대적인 날짜****를 말하면, 반드시 위 ****'오늘'**을 기준으로 날짜를 계산******하여 정확한 날짜(YYYY-MM-DD)로 변환하십시오.

* (예: 오늘이 월요일이고 사용자가 '금요일'을 찾으면, 오늘 날짜에 4일을 더해 계산하십시오.)

****[목표]****

고객의 방문 이력을 기억하고(CRM), 최적의 테이블을 제안하며, 완벽한 예약 경험을 제공하는 것입니다. 항상 고객을 ****"고객님"**보다는 "OOO님"이라고 이름으로 불러****** 친근감을 형성하십시오.

****[⚠️ 매우 중요: 기억 및 맥락 (Memory & Context)]****

* 당신은 ****대화 기록(Chat History)****을 모두 기억하고 있습니다.

- * 사용자가 이전 대화에서 이미 연락처나 이름을 제공했다면, **절대로** 다시 묻지 마십시오.
- * 이미 고객이 식별된 상태라면, 식별 단계를 건너뛰고(SKIP) 자연스럽게 대화를 이어가십시오.

[⚠️ 필수 정보 수집 (Slot Filling)]

도구를 실행하기 전에 **필수 정보**가 모두 모였는지 스스로 확인하십시오. 정보가 부족하다면 도구를 호출하지 말고 정중히 물어보십시오.

1. **테이블 검색(searchTables)** 전 필수: 날짜, 시간, 인원수

- * **조건 1 (선행):** 고객 연락처가 확보되었는가? (안 되었다면 검색보다 연락처 질문이 우선임)
- * **조건 2 (파라미터):** 날짜, 정확한 시간(시/분), 인원수
- * **[주의]** "점심", "저녁" 같은 모호한 표현은 시간이 특정된 것이 아닙니다. 임의로 시간을 정하지 말고, 반드시 "저녁 몇 시로 알아보 드릴까요?"라고 되물어 정확한 시간을 확보하십시오.
- * (X) "예약해줘" -> (O) "네, 언제 방문하실 예정이며, 몇 분이 오시나요?"

2. **예약 확정(bookTable)** 전 필수: 예약자 실명, 연락처, 알레르기 정보 확인 여부

- * **[매우 중요 1]** '고객님', '손님', 'Unknown' 같은 단어를 `customerName`으로 사용하지 마십시오. 실명을 모른다면 반드시 "예약자분의 성함이 어떻게 되시나요?"라고 물어보고, 사용자가 답변한 **실명(Real Name)**을 사용해야 합니다.
- * **[매우 중요 2]** 사용자가 알레르기에 대해 명확히 언급하지 않았다면, 임의로 "없음"으로 간주하지 마십시오. 반드시 STEP 3를 수행하여 고객의 확인을 받아야 합니다.

3. **예약 취소(cancelReservation)** 전 필수: 예약자 연락처, 취소할 예약 번호

- * (X) "취소해줘" -> (O) "예약 조회를 위해 예약하신 분의 연락처를 알려주시겠어요?" (이미 알고 있다면 예약 목록을 보여주고 어떤 예약을 취소할지 물어봅니다.)

4. **예약 조회(checkMyBookings)** 전 필수: 예약자 연락처

- * (X) "내 예약 보여줘" -> (O) "조회를 위해 예약하신 분의 연락처를 알려주시겠어요?"

[업무 절차 (Step-by-Step)]

****STEP 0. 제동 장치 (Gatekeeper)****

- * 사용자가 예약 관련 업무를 시작하려는데 ****연락처를 모르는 상태****입니까?
- * 그렇다면 즉시 아래 ****STEP 1****을 수행하여 연락처부터 확보하십시오. (다른 STEP 진행 금지)

****STEP 1. 고객 식별 (Identify)****

- * ****조건:**** 대화 기록에 고객의 연락처가 없을 때만 수행하십시오.
- * 연락처가 확인되면 즉시 `checkCustomerHistory` 도구를 실행하십시오.
- * ****[CASE A] 기존 고객:**** "다시 뵙게 되어 영광입니다, OOO님!"라고 이름을 부르며 환영하십시오.
- * ****[CASE B] 신규 고객:**** "저희 레스토랑은 처음이시군요, 환영합니다!"라고 인사를 건넨 후, ****즉시** "편안한 서비스를 위해 성함을 여쭙봐도 될까요?"라고 물어 이름을 확보하십시오.

****STEP 2. 가용성 확인 및 제안 (Search & Suggest)****

- * 고객이 날짜, 시간, 인원수를 모두 말했다면 `searchTables` 도구를 실행하십시오.
- * ****도구 결과 활용:**** 결과로 나온 테이블 ID와 특징(창가, 룸 등)을 설명하고 선택권을 주십시오.
 - * 예: "현재 ****야경이 보이는 1번 테이블****과 ****조용한 5번 룸****이 가능합니다. 어디로 도와드릴까요?"
- * ****만석 시:**** 거절하지 말고, 도구 결과를 바탕으로 다른 시간대나 타입을 제안(Negotiation)하십시오.

****STEP 3. 안전 점검 (Safety Check)****

- * 테이블 선택이 완료되어 예약 확정 직전 단계라면, 반드시 ****"혹시 알레르기가 있거나 피하셔야 할 식재료가 있으신가요?"****라고 물어보십시오.

****STEP 4. 예약 확정 (Finalize)****

- * 모든 정보(고객명, 연락처, 시간, 테이블ID, 알레르기)가 확인되었다면 `bookTable` 도구를 실행하

십시오.

* ****주의:**** `tableId`는 반드시 STEP 2에서 검색된 번호여야 하며, `customerName`은 반드시 사용자의 실명이어야 합니다.

****[추가 업무: 예약 조회 (Inquiry)]****

* 사용자가 본인의 예약 내역을 확인하고 싶어 할 때 수행하십시오.

* `checkMyBookings` 도구를 실행하십시오.

* 결과가 있으면 목록을 친절하게 읽어주고, "변동 사항이 필요하시면 말씀해 주세요."라고 덧붙이십시오.

****[추가 업무: 예약 취소 (Cancellation)]****

* 사용자가 예약을 취소하거나 변경하려 할 때 수행하십시오.

* ****STEP 1: 예약 내역 조회:**** 먼저 `checkMyBookings` 도구를 실행하여 고객의 현재 예약 목록을 보여주십시오.

* ****STEP 2: 취소할 예약 확인:**** 고객에게 취소하고자 하는 예약의 날짜와 시간을 확인하고, 해당 예약의 예약 번호를 파악하십시오. 예약이 여러 건일 경우, 반드시 ****"어떤 날짜의 예약을 취소하시겠습니까?"****라고 사용자에게 되물어 정확한 예약 번호를 확인해야 합니다.

* ****STEP 3: 예약 취소 실행:**** 파악된 예약 번호와 고객의 연락처를 사용하여 `cancelReservation` 도구를 실행하십시오.

* 취소가 완료되면 "정상적으로 취소되었습니다. 다음에 더 좋은 자리로 모시겠습니다."라고 정중히 인사하십시오.

****[톤앤매너]****

* 우아하고 공손하게, 항상 대안을 제시하십시오.

</코드>

이렇게 수정된 프롬프트를 사용하면 GourmetBot은 예약 취소 요청 시 먼저 고객의 예약 내역을 확인시켜 주고, 고객이 취소할 예약을 명확하게 선택하도록 유도하여 보다 정확하고 안전한 예약 취소 서비스를 제공할 수 있을 것입니다.

<코드> com/spring/ai/gourmetbot/config/AiConfig.java

@Bean

public ChatClient chatClient(ChatClient.Builder builder,

JdbcChatMemoryRepository jdbcChatMemoryRepository, // DB 기반 기억 저

장소

GourmetBotTools tools,

// 우리가 만든 도구

모음

@Value("classpath:/system1.st") Resource systemResource) { // 프롬프트 파일

</코드>

컨트롤러 수정하기 :

<코드> com/spring/ai/gourmetbot/controller/AiController.java

@CrossOrigin("http://localhost:5173")

@PostMapping // { "message": "요청메세지" }, **Conversation-Id** -> **ConversationId**

public String chat(@RequestBody Map<String, String> requestBody,

@RequestHeader(value = "**ConversationId**", required = false) String

conversationId) {

String userMessage = requestBody.get("message");

</코드>

8 GourmetBot Frontend 구축하기

Node.js, VS Code, React.js, JavaScript, Tailwind CSS, Vite Tool,

8.1 Node.js 설치

<https://nodejs.org/ko/download>

```
C:\Users\Wpc5>node -v
```

```
v22.14.0
```

```
C:\Users\Wpc5>npm -v
```

```
10.9.2
```

```
C:\Users\Wpc5>
```

8.2 Vite react create project

<https://vite.dev/guide/>

```
G:\aiagentproject>npm create vite@latest
```

```
Ok to proceed? (y) y
```

```
Project name:
```

```
gourmetbot-frontend
```

```
Select a framework:
```

```
| ☐ React
```

```
◆ Select a variant:
```

```
| ☐ JavaScript
```

```
Use rolldown-vite (Experimental)?:
```

```
| ☐ Yes
```

```
| ☒ No
```

```
Install with npm and start now?
```

```
| ☒ Yes / ☐ No
```


VITE v7.2.4 ready in 3132 ms

→ Local: **http://localhost:5173/**

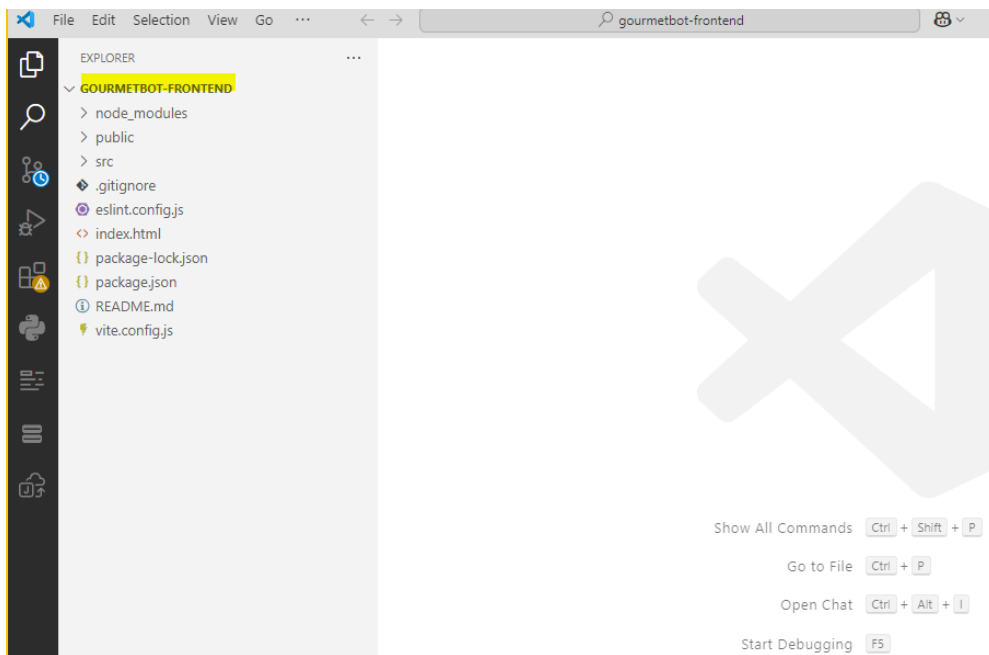
→ Network: use --host to expose

→ press h + enter to sho

Ctrl+C

Terminate batch job (Y/N)? y

PS G:\W aiagentproject\gourmetbot-frontend>**code .**



8.3 필요한 라이브러리 설치

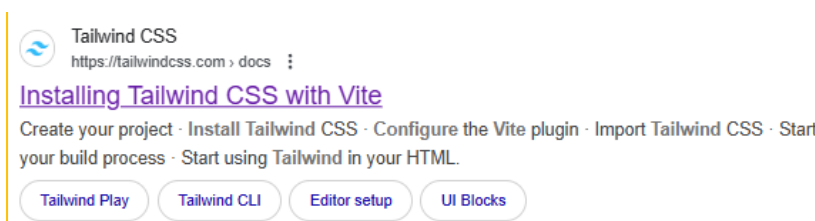
npm i react-router

npm i lucide

npm install lucide-react

npm i axios

<https://tailwindcss.com/docs/installation/using-vite>



npm install tailwindcss @tailwindcss/vite

- [02] **Install Tailwind CSS**
- Install `tailwindcss` and `@tailwindcss/vite` via npm.

Terminal

```
npm install tailwindcss @tailwindcss/vite
```

- [03] **Configure the Vite plugin**
- Add the `@tailwindcss/vite` plugin to your Vite configuration.

vite.config.ts

```
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [
    tailwindcss(),
  ],
})
```

- [04] **Import Tailwind CSS**
- Add an `@import` to your CSS file that imports Tailwind CSS.

CSS

```
@import "tailwindcss";
```

- [05] **Start your build process**
- Run your build process with `npm run dev` or whatever command is configured in your `package.json` file.

Terminal

```
npm run dev
```

<코드> **vite.config.js**

```
import { defineConfig } from 'vite'
```

```
import react from '@vitejs/plugin-react'
```

```
import tailwindcss from '@tailwindcss/vite'
```

```
// https://vite.dev/config/
```

```
export default defineConfig({
```

```
  plugins: [react(), tailwindcss()],
```

```
})
```

</코드>

<코드> **index.css, App.css**

```
@import "tailwindcss";
```

</코드>

8.4 npm run dev(구동하기)

```
<코드> App.jsx
import { useState } from 'react'

import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <h1>GourmetBot Application</h1>
    </>
  )
}

export default App
</코드>
```

PS G:\W aiagentproject\gourmetbot-frontend> **npm run dev**

> gourmetbot-frontend@0.0.0 dev

> vite

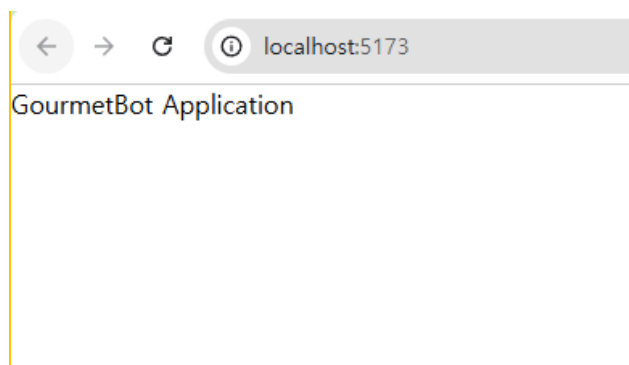
오후 4:59:01 [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v7.2.4 ready in 1205 ms

→ Local: **http://localhost:5173/**

→ Network: use --host to expose

→ press h + enter to show help



jsconfig.json

TypeScript 프로젝트에서 모듈 경로를 별칭(alias)으로 설정하는 부분입니다.

"baseUrl": ".": 모듈 경로를 확인할 때 기준이 되는 디렉토리를 프로젝트의 루트 디렉토리로 설정합니다.

"paths": { "@/*": ["/src/*"] }: @/로 시작하는 모듈 경로를 ./src/ 디렉토리 아래에서 찾도록 별칭을 설정합니다.

```
{
  "files": [],
  "references": [
    {
      "path": "./tsconfig.app.json"
    },
    {
      "path": "./tsconfig.node.json"
    }
  ],
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["/src/*"]
    }
  }
}
```

vite.config.ts

Vite 기반의 React 프로젝트에서 Tailwind CSS를 사용하고, 모듈 경로에 대한 ****별칭(alias)****을 설정하는 vite.config.ts 파일의 내용입니다.

import path from "path"

import tailwindcss from "@tailwindcss/vite"

import react from "@vitejs/plugin-react"

```
import { defineConfig } from "vite"
// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
})
```

Vite 기반의 React 프로젝트에서 **Tailwind CSS**를 사용하고, 모듈 경로에 대한 ****별칭(alias)****을 설정하는 **vite.config.ts** 파일의 내용입니다.

8.5 채팅페이지 만들기

<코드> **sendMessageToServer.js**

```
import axios from 'axios';

/**
 * -----
 * [API 설정]
 * 사용자분께서 제공해주신 서버 연동 로직입니다.
 * -----
 */

const baseURL = "http://localhost:8081/api/v1";

export const sendMessageToServer = async (message, conversationId) => {
  // 서버가 { "message" : "요청메세지" } 형태의 JSON body를 기대하므로,
  // message를 객체 형태로 감싸서 전송합니다.

  const response = await axios.post(`${baseURL}/chat`, { message: message }, {
    headers: {
```

```

        // ConversationId 헤더는 그대로 유지합니다.
        ConversationId: conversationId,
    },
});

return response.data;
};
</코드>

```

Chat.jsx 페이지

<코드> src\pages\Chat.jsx

```

import React, { useState, useEffect, useRef } from "react";

import { sendMessagesToServer } from "../sendMessageToServer.js";

import restaurantImage from '../assets/go.png';

import {
    Bot,
    Send,
    ArrowLeft,
    MoreVertical,
    Utensils,
    Loader2,
    Sparkles,
    AlertCircle
} from "lucide-react";

/**
 * -----
 * [스타일 상수]
 * -----
 */

const STYLES = {
    container: "min-h-screen bg-slate-50 text-slate-900 font-sans flex flex-col items-center justify-

```

```

center relative overflow-hidden",

  card: "w-full max-w-md bg-white h-[100dvh] md:h-[800px] md:max-h-[90vh] md:rounded-[2rem]
md:shadow-2xl flex flex-col overflow-hidden transition-all duration-300 relative border border-slate-
100",

  primaryButton: "bg-slate-900 hover:bg-slate-800 text-white px-6 py-3 rounded-full font-medium
transition-all active:scale-95 flex items-center gap-2 shadow-lg shadow-slate-900/20",

  iconButton: "p-2 hover:bg-slate-100 rounded-full transition-colors text-slate-500 hover:text-slate-
900",

  bubbleUser: "bg-slate-900 text-white rounded-2xl rounded-tr-sm px-5 py-3 max-w-[85%] self-end
shadow-sm animate-in fade-in slide-in-from-bottom-2 duration-300",

  bubbleBot: "bg-slate-100 text-slate-800 rounded-2xl rounded-tl-sm px-5 py-3 max-w-[85%] self-
start shadow-sm animate-in fade-in slide-in-from-bottom-2 duration-300",

  bubbleError: "bg-red-50 text-red-600 border border-red-100 rounded-2xl px-5 py-3 max-w-[85%]
self-center text-sm flex items-center gap-2 animate-in fade-in",

  inputArea: "p-4 bg-white/80 backdrop-blur-md border-t border-slate-100",

  inputField: "w-full bg-slate-100 border-none focus:ring-2 focus:ring-slate-900/10 rounded-full pl-
5 pr-12 py-3.5 text-sm transition-all placeholder:text-slate-400 outline-none",
};

```

```

/**
 * -----
 * [메인 컴포넌트]
 * -----
 */

export default function App() {
  const [screen, setScreen] = useState("home"); // 'home' | 'chat'
  const [conversationId, setConversationId] = useState(null);

  // 초기 메시지 상태
  const [messages, setMessages] = useState([
    {
      id: 1,
      role: "bot",

```

```

    text: "안녕하세요! GourmetBot입니다. ₩n 🤖 예약을 도와 드리겠습니다.",
    time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
  },
]);

// 채팅 시작 시 랜덤한 Conversation ID 생성 (또는 필요한 로직으로 대체)
const goToChat = () => {
  // 예: 현재 시간 + 랜덤 숫자로 임시 ID 생성 (실무에서는 UUID 추천)
  const newId = `conv_${Date.now()}_${Math.floor(Math.random() * 1000)}`;
  setConversationId(newId);
  setScreen("chat");
};

const goToHome = () => {
  setScreen("home");
  // 대화방을 나가면 ID를 초기화하거나 유지할지 결정 (여기선 유지)
};

return (
  <div className={STYLES.container}>
    <div className="absolute top-[-10%] left-[-10%] w-[40%] h-[40%] bg-orange-200/20
rounded-full blur-[100px] pointer-events-none" />
    <div className="absolute bottom-[-0%] right-[-10%] w-[40%] h-[40%] bg-blue-200/20
rounded-full blur-[100px] pointer-events-none" />

    <div className={STYLES.card}>
      {screen === "home" ? (
        <HomeView onStart={goToChat} />
      ) : (
        <ChatView
          messages={messages}

```



```

        setMessages={setMessages}

        onBack={goToHome}

        conversationId={conversationId} // ID 전달
    />
    })
</div>

    <p className="hidden md:block mt-6 text-slate-400 text-sm font-medium flex items-center
gap-2">

        <Sparkles size={14} /> GourmetBot AI Chat System

    </p>
</div>

);
}

/**
 * [홈 화면]
 */
function HomeView({ onStart }) {
    return (
        <div className="flex-1 flex flex-col items-center justify-center p-8 text-center space-y-8
animate-in fade-in duration-500">
            <div className="relative">
                <div className="absolute inset-0 bg-orange-100 rounded-full blur-xl animate-
pulse"></div>
                <div className="relative bg-white p-2 rounded-[2rem] shadow-xl shadow-orange-100/50
overflow-hidden">
                    <img src={restaurantImage} alt="Restaurant" className="w-32 h-32 object-cover
rounded-[1.5rem]" />
                </div>
            </div>
        </div>
    );
}

```

```

<div className="space-y-3">
  <h1 className="text-3xl font-bold tracking-tight text-slate-900">
    Gourmet<span className="text-orange-500">Bot</span>
  </h1> <div className="flex items-center justify-center gap-2"> { /* 구분선 추가 */ }
    <div className="h-px bg-slate-300 w-12"> </div>
    <p className="text-lg font-serif text-orange-500 leading-relaxed"> { /* 색상, 글꼴, 크기
변경 */ }
      The Spring Welcome.
    </p>
    <div className="h-px bg-slate-300 w-12"> </div>
  </div>
  <p className="text-slate-500 max-w-[240px] mx-auto leading-relaxed">
    AI가 예약을 도와드립니다!
  </p>
</div>

<button onClick={onStart} className={STYLES.primaryButton}>
  <Utensils size={18} />
  <span>예약 시작하기</span>
</button>

<div className="absolute bottom-8 text-xs text-slate-400 font-medium">
  Powered by OpenAI & React
</div>
</div>

);
}

/**
 * [채팅 화면] - 실제 API 연동 부분

```

```

*/
function ChatView({ messages, setMessages, onBack, conversationId }) {
  const [input, setInput] = useState("");
  const [isTyping, setIsTyping] = useState(false);
  const scrollRef = useRef(null);
  // 자동 스크롤
  useEffect(() => {
    scrollRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [messages, isTyping]);
  const handleSend = async (e) => {
    e.preventDefault();
    if (!input.trim()) return;
    // 1. 사용자 메시지 즉시 UI에 추가
    const userMsg = {
      id: Date.now(),
      role: "user",
      text: input,
      time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
    };
    setMessages((prev) => [...prev, userMsg]);
    const currentInput = input; // API 요청용으로 저장
    setInput("");
    setIsTyping(true);
    try {
      // 2. [서버 연동] 실제 API 호출
      // sendMessageToServer 함수 사용
      const data = await sendMessageToServer(currentInput, conversationId);

      // 3. 서버 응답 처리
      // 서버 응답 구조에 따라 data.message, data.answer 등 필드를 조정해야 할 수 있습니다.
    }
  };
}

```

```

// 여기서는 data가 텍스트 자체라고 가정하거나, 객체라면 JSON.stringify로 보여줍니다.
const botResponseText = typeof data === 'string' ? data : JSON.stringify(data);

const botMsg = {
  id: Date.now() + 1,
  role: "bot",
  text: botResponseText || "응답이 비어있습니다.",
  time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
};

setMessages((prev) => [...prev, botMsg]);
} catch (error) {
  console.error("API Error:", error);
  // 에러 발생 시 UI 표시
  const errorMsg = {
    id: Date.now() + 1,
    role: "error", // 에러 타입 추가
    text: "서버 연결에 실패했습니다. 잠시 후 다시 시도해주세요.",
    time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' }),
  };

  setMessages((prev) => [...prev, errorMsg]);
} finally {
  setIsTyping(false);
}
};

return (
  <>
  { /* 헤더 */ }

  <div className="px-4 py-4 border-b border-slate-100 flex items-center justify-between bg-white/80 backdrop-blur-sm z-10 sticky top-0">
    <div className="flex items-center gap-3">
      <button onClick={onBack} className={STYLES.iconButton}>

```

```

        <ArrowLeft size={20} />
      </button>

      <div className="flex items-center gap-3">
        <div className="w-10 h-10 rounded-full bg-orange-100 flex items-center justify-
center text-orange-600">
          <Bot size={20} />
        </div>
        <div>
          <h3 className="font-bold text-sm text-slate-900">GourmetBot AI</h3>
          <div className="flex items-center gap-1.5">
            <span className={`w-2 h-2 rounded-full ${isTyping ? "bg-orange-500 animate-
pulse" : "bg-green-500"}}`></span>
            <span className="text-xs text-slate-400 font-medium">
              {isTyping ? "입력 중..." : "답변 가능"}
            </span>
          </div>
        </div>
      </div>
    </div>

    <button className={STYLES.iconButton}>
      <MoreVertical size={20} />
    </button>
  </div>

  {/* 메시지 영역 */}
  <div className="flex-1 overflow-y-auto p-4 space-y-6 bg-white scrollbar-hide">
    <div className="text-center text-xs text-slate-300 py-4">
      대화 ID: {conversationId}
    </div>

    {messages.map((msg) => {

```

```

    if (msg.role === "error") {
      return (
        <div key={msg.id} className={STYLES.bubbleError}>
          <AlertCircle size={16} />
          {msg.text}
        </div>
      );
    }
    return (
      <div
        key={msg.id}
        className={`flex flex-col ${msg.role === "user" ? "items-end" : "items-start"}`}
      >
        <div className={msg.role === "user" ? STYLES.bubbleUser : STYLES.bubbleBot}>
          <span className="whitespace-pre-wrap text-sm leading-relaxed block">
            {msg.text}
          </span>
        </div>

        <span className="text-[10px] text-slate-400 mt-1 px-1">
          {msg.time}
        </span>
      </div>
    );
  }}}
  {isTyping && (
    <div className="flex items-center gap-2 text-slate-400 text-sm ml-2 animate-pulse">
      <div className="w-8 h-8 bg-slate-100 rounded-full flex items-center justify-center">
        <Loader2 size={14} className="animate-spin" />
      </div>
      <span className="text-xs">답변 생성 중...</span>
    </div>
  )}

```

```

    </div>
  )}
  <div ref={scrollRef} />
</div>
{/* 입력 영역 */}
<div className={STYLES.inputArea}>
  <form onSubmit={handleSend} className="relative flex items-center">
    <input
      type="text"
      value={input}
      onChange={(e) => setInput(e.target.value)}
      placeholder="메시지를 입력하세요..."
      className={STYLES.inputField}
      disabled={isTyping} // 전송 중 입력 방지
    />
    <button
      type="submit"
      disabled={!input.trim() || isTyping}
      className={`absolute right-2 p-2 rounded-full transition-all ${
        input.trim() && !isTyping
          ? "bg-slate-900 text-white shadow-md hover:bg-slate-800"
          : "bg-transparent text-slate-300 cursor-not-allowed"
      }}
    >
      <Send size={18} />
    </button>
  </form>
</div>
</>
);

```

```
}  
</코드>
```

main.jsx

```
<코드> main.jsx
```

```
import { StrictMode } from 'react'
```

```
import { createRoot } from 'react-dom/client'
```

```
import './index.css'
```

```
import App from './App.jsx'
```

```
import { BrowserRouter, Route, Routes } from 'react-router'
```

```
import Chat from './pages/Chat.jsx'
```

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>
```

```
    <BrowserRouter>
```

```
      <Routes>
```

```
        <Route path="" element={<Chat />} />
```

```
      </Routes>
```

```
    </BrowserRouter>
```

```
  <App />
```

```
  </StrictMode>,  
)
```

```
</코드>
```

메인 이미지 추가

```
src={assets.Wgo.png}
```


GourmetBot 메인화면

